

INFORMATIQUE • 1ÈRE TI

MODULE 2 : PROGRAMMATION

## LEÇON N05

PROGRAMMATION WEB  
STATIQUE EN HTML

DURÉE CONSEILLÉE : 10 H

## DANS CETTE LEÇON

Cette leçon initie l'élève à la programmation web statique en HTML : structure d'une page, balises, attributs, hyperliens, médias, tableaux et formulaires. Elle aboutit à la création d'un mini-site web organisé et fait le point sur les limites du HTML.

- À la fin de cette leçon, je sais définir un site web, une balise, un attribut, un hyperlien, une URL et un formulaire.
- À la fin de cette leçon, je sais structurer une page HTML valide avec les caractères accentués (<meta charset="utf-8">).
- À la fin de cette leçon, je sais insérer des listes imbriquées et des liens hypertextes (relatifs ou absolus) avec infobulle.
- À la fin de cette leçon, je sais insérer des médias (image, image cliquable, son, vidéo) à l'aide d'une URL.
- À la fin de cette leçon, je sais créer un tableau, régler ses attributs et fusionner des cellules (colspan, rowspan).
- À la fin de cette leçon, je sais construire un formulaire complet (champs de saisie, boutons radio, cases à cocher, liste déroulante, boutons).

TÉLÉCHARGE  
ONBUCH

Gratuit • Google Play  
Tuteur IA, annales,  
concours, résultats



## PASSE À ONBUCH+

Tous les cours signés, Tuteur IA  
illimité, fiches PDF — onglet  
OnBuch+ de l'app

## CE COURS SIGNÉ CONTIENT

## COURS

complet, illustré, pas à  
pas

## MÉTHODES

et exercices résolus

## EXERCICES

type examen + corrigés

## FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.  
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

**SOMMAIRE**

1. Site web, HTML et structure d'une page
2. Listes, hyperliens et URL
3. Insertion de médias : images, sons, vidéos
4. Les tableaux en HTML
5. Les formulaires
6. Organisation d'un site, balises de section et limites du HTML
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

**OBJECTIFS DE LA LEÇON**

- À la fin de cette leçon, je sais définir un site web, une balise, un attribut, un hyperlien, une URL et un formulaire.
- À la fin de cette leçon, je sais structurer une page HTML valide avec les caractères accentués (`<meta charset="utf-8">`).
- À la fin de cette leçon, je sais insérer des listes imbriquées et des liens hypertextes (relatifs ou absolus) avec infobulle.
- À la fin de cette leçon, je sais insérer des médias (image, image cliquable, son, vidéo) à l'aide d'une URL.
- À la fin de cette leçon, je sais créer un tableau, régler ses attributs et fusionner des cellules (`colspan`, `rowspan`).
- À la fin de cette leçon, je sais construire un formulaire complet (champs de saisie, boutons radio, cases à cocher, liste déroulante, boutons).
- À la fin de cette leçon, je sais organiser les dossiers d'un site web et structurer une page avec `<DIV>`, `<SECTION>` et `<SPAN>`.
- À la fin de cette leçon, je sais énoncer les limites du HTML et le rôle complémentaire du CSS et du JavaScript.

**PRÉREQUIS**

- Notion de fichier, de dossier et d'arborescence (organisation d'un disque dur)
- Notion d'adresse URL et de navigation sur Internet (classes précédentes)

- Utilisation d'un éditeur de texte simple (Bloc-notes, Notepad++) et d'un navigateur web
- Notion de langage et de syntaxe (Module 1 : algorithmique)

## 1 SITE WEB, HTML ET STRUCTURE D'UNE PAGE

Au lycée de Nkongsamba, le proviseur a une idée ambitieuse : publier sur Internet les résultats du probatoire et proposer un formulaire d'inscription en ligne pour les nouveaux élèves. Fini les longues files d'attente devant le tableau d'affichage ! Il confie la mission à un élève de Première TI. Celui-ci, plein d'enthousiasme, ouvre Microsoft Word, tape le texte, insère un tableau... et envoie le fichier. Résultat : impossible à afficher correctement dans un navigateur, aucun lien ne fonctionne, le formulaire n'existe pas. Il découvre alors une vérité fondamentale : le Web possède son propre langage, le **HTML**, et c'est lui seul que les navigateurs comprennent. Comment structurer une page web ? Comment y insérer des images, des tableaux de résultats et un formulaire d'inscription ? C'est toute la problématique de cette leçon : **créer un site web statique en HTML**. Commençons par le commencement : comprendre ce qu'est un site web et comment une page HTML est construite.

### 1.1 Site web, page web, navigateur et serveur

### 1.2 Les acteurs du Web : définitions

Avant d'écrire la moindre ligne de code, il faut comprendre le décor dans lequel notre page va vivre. Quand tu tapes une adresse dans ton téléphone au cybercafé, que se passe-t-il exactement ?

#### DÉFINITION Site web et page web

Une **page web** est un document informatique, écrit dans un langage de description (le HTML), consultable à l'aide d'un navigateur. Un **site web** est un ensemble cohérent de pages web reliées entre elles par des liens hypertextes, hébergées sur un serveur et accessibles via une même adresse (nom de domaine).

#### DÉFINITION Navigateur et serveur web

Le **navigateur** (ou browser) est le logiciel client qui demande, reçoit et interprète les pages web pour les afficher à l'écran : Google Chrome, Mozilla Firefox, Microsoft Edge, Opera... Le **serveur web** est un ordinateur (et le logiciel qui y tourne, comme Apache ou Nginx) qui stocke les fichiers du site et les envoie aux navigateurs qui les réclament, selon le protocole HTTP.

Le dialogue est donc toujours le même : le navigateur demande une page au serveur, le serveur envoie le fichier HTML, le navigateur interprète le code et affiche le résultat. Ce mécanisme s'appelle le modèle **client-serveur**.

**DÉFINITION** Site statique et site dynamique

Un site **statique** est un site dont le contenu des pages est fixe : le fichier HTML envoyé par le serveur est identique pour tous les visiteurs et ne change que si le concepteur modifie le code à la main. Un site **dynamique** génère ses pages à la volée, souvent à partir d'une base de données (exemples : Facebook, un site d'actualités, les résultats du Probatoire en ligne). En Première TI, nous créons des sites statiques : le dynamique côté serveur viendra en Terminale.

**EXEMPLE** Statique ou dynamique ?

- La page de présentation du lycée de Nkongsamba (historique, photos, contact) : site statique, car le contenu change rarement.
- La consultation des notes en ligne, où chaque élève voit ses propres résultats : site dynamique, car la page affichée dépend du visiteur.

### 1.3 Le langage HTML : un langage à balises interprété

HTML signifie HyperText Markup Language, c'est-à-dire « langage de balisage hypertexte ». Ce n'est **pas** un langage de programmation comme le C : il ne contient ni variables, ni boucles, ni calculs. C'est un langage de description : il indique au navigateur la structure du document (« ceci est un titre, ceci est un paragraphe, ceci est une image »).

**PROPRIÉTÉ** HTML est interprété, pas compilé

Contrairement au langage C qui est compilé (traduit une fois pour toutes en langage machine par un compilateur), le code HTML est **interprété** : le navigateur lit le fichier texte ligne par ligne et affiche le résultat au fur et à mesure. Conséquences pratiques :

- aucun outil spécial n'est nécessaire : un simple éditeur de texte (Bloc-notes, Notepad++) suffit ;
- une erreur dans le code ne « plante » pas la page : le navigateur ignore la balise inconnue et continue l'affichage ;
- le code source de n'importe quelle page est visible (clic droit → « Afficher le code source »).

## 1.4 Balises et attributs : les briques du HTML

### 1.5 La balise : ouvrante, fermante, orpheline

Pour marquer un contenu, on l'entoure de **balises**. Une balise est un mot-clé écrit entre les signes `<` et `>`.

#### DÉFINITION Balise

Une **balise** est un marqueur encadré par les chevrons `<` et `>` qui indique au navigateur la nature d'un contenu. La plupart des balises fonctionnent par paire :

- la balise ouvrante : `<p>` marque le début ;
- la balise fermante : `</p>` (avec un slash `/`) marque la fin.

Certaines balises, dites **orphelines** (ou auto-fermantes), n'ont pas de contenu à encadrer et s'écrivent seules : `<br>` (retour à la ligne), `<hr>` (ligne horizontale), `<img>` (image), `<meta>` (métadonnée).

#### EXEMPLE Balise paire et balises orphelines

```
<p>Le lycée de Nkongsamba est situé au pied du Mont Manengouba.</p>  
<hr>  
<p>Deuxième ligne après un retour :<br>voici la suite.</p>
```

Ici, `<p>` encadre un paragraphe, `<hr>` trace une ligne horizontale et `<br>` force un retour à la ligne à l'intérieur du paragraphe.

#### ATTENTION Pièges classiques avec les balises

- Oublier la balise fermante : `<b>Important sans </b>` met toute la suite de la page en gras.
- Mal imbriquer : `<b><i>texte</b></i>` est incorrect ; la dernière balise ouverte doit être la première fermée : `<b><i>texte</i></b>`.
- Majuscules/minuscules : HTML est insensible à la casse (`<P>` fonctionne), mais la convention professionnelle est d'écrire en minuscules.

### 1.6 L'attribut de balise

Une balise seule donne une information générale ; un **attribut** la précise. Par exemple, `<img>` ne suffit pas : il faut dire quelle image afficher.

**DÉFINITION** Attribut de balise

Un **attribut** est une information complémentaire placée dans la balise ouvrante, sous la forme `nom="valeur"`. La valeur est toujours écrite entre guillemets. Certains attributs sont obligatoires (la balise ne fonctionne pas sans eux), d'autres sont facultatifs.

**PROPRIÉTÉ** Syntaxe générale d'une balise avec attributs

```
<balise attribut1="valeur1" attribut2="valeur2"> contenu </balise>
```

Exemples d'attributs obligatoires : `src` et `alt` pour `<img>`, `href` pour le lien `<a>`. Exemples d'attributs facultatifs : `width`, `height`, `align`, `title` (infobulle), `id`.

**EXEMPLE** Attributs obligatoires et facultatifs

```

```

`src` (chemin de l'image) et `alt` (texte alternatif affiché si l'image ne se charge pas) sont obligatoires ; `width` (largeur en pixels) est facultatif.

## 1.7 La structure minimale d'une page HTML

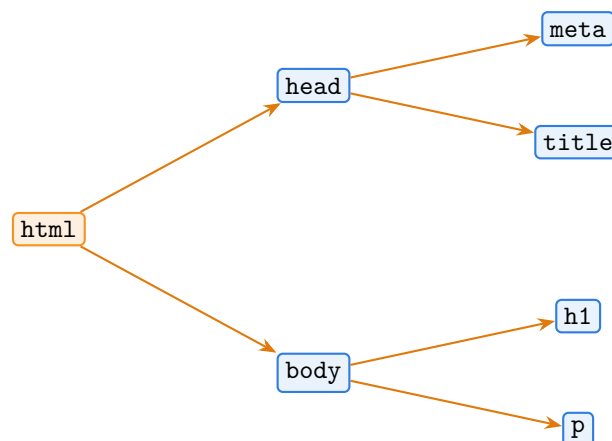
Toute page HTML bien formée possède un squelette immuable. Apprends-le par cœur : c'est la charpente de ta maison.

**PROPRIÉTÉ** Structure obligatoire d'un document HTML

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Titre de la page</title>
  </head>
  <body>
    <!-- Contenu visible de la page -->
  </body>
</html>
```

- `<!DOCTYPE html>` : déclaration placée en toute première ligne, indiquant au navigateur qu'il s'agit d'un document HTML5 ;

- `<html>` : élément racine qui englobe tout le document ;
- `<head>` : en-tête, contient les informations invisibles sur la page (encodage, titre, liens vers les fichiers CSS) ;
- `<title>` : titre affiché dans l'onglet du navigateur (obligatoire, placé dans `<head>`) ;
- `<body>` : corps de la page, contient tout ce qui sera affiché à l'écran.



Arborescence d'un document HTML : chaque balise est un nœud, `html` est la racine, `head` et `body` ses deux enfants.

#### **ATTENTION** Les caractères accentués : le piège du charset

Si tu oublies la ligne `<meta charset="utf-8">` dans le `<head>`, les caractères accentués s'affichent de façon corrompue : le mot « élève » devient « Ã©lève », « Résultats » devient « RÃ©sultats ». C'est le bug le plus fréquent chez les débutants camerounais, car le français est plein d'accents ! La solution est systématique : toujours déclarer `<meta charset="utf-8">` en premier dans le `<head>`, et enregistrer le fichier en encodage UTF-8 dans l'éditeur.

## 1.8 Les balises de texte de base

Dans le `<body>`, on structure le texte avec des balises simples :

| Balise      | Rôle                                                                |
|-------------|---------------------------------------------------------------------|
| <h1> à <h6> | Titres, du plus important (h1, gros) au moins important (h6, petit) |
| <p>         | Paragraphe (avec un espacement automatique avant et après)          |
| <br>        | Retour à la ligne simple, sans nouveau paragraphe (orpheline)       |
| <hr>        | Ligne horizontale de séparation (orpheline)                         |
| <b>         | Texte en gras (bold)                                                |
| <i>         | Texte en italique                                                   |

**ATTENTION** Titre n'est pas gras

<h1> n'est pas un « gros texte décoratif » : c'est un titre structurant. Une page ne devrait contenir qu'un seul <h1> (le titre principal), puis des <h2> pour les grandes parties, etc. Et pour un simple retour à la ligne dans une adresse, utilise <br>, pas <p>, qui crée un nouveau paragraphe avec un espacement.

**EXEMPLE** Code complet : la page « Mon lycée »

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Lycee de Nkongsamba</title>
</head>
<body>
  <h1>Bienvenue au Lycée de Nkongsamba</h1>
  <hr>
  <h2>Présentation</h2>
  <p>Notre <b>lycée</b> est situé au pied du Mont Manengouba,
  dans la région du <i>Littoral</i>. <br>
  Il accueille plus de 3000 élèves.</p>
  <h2>Résultats au probatoire</h2>
  <p>Le taux de réussite 2024 est de 78 \%.</p>
</body>
</html>
```

Observe : le charset est déclaré, les accents s'afficheront correctement ; un seul <h1>, deux <h2> ; <hr> sépare l'en-tête du contenu ; <br> fait un retour à la ligne à l'intérieur du paragraphe.

**Code source (fichier .html)**

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Mon lycée</title>
</head>
<body>
  <h1>Bienvenue</h1>
  <p>Lycee de
    Nkongsamba</p>
</body>
</html>
```

interprétation

**Rendu dans le navigateur**

**Bienvenue**  
Lycée de Nkongsamba

Le navigateur interprète le code source (à gauche) et affiche le rendu (à droite) : les balises disparaissent, seule la structure reste visible.

## 1.9 Méthode de travail : créer et afficher sa première page

**MÉTHODE** Créer sa première page HTML en 5 étapes

1. Ouvrir un éditeur de texte : Bloc-notes (Windows), Notepad++ ou tout autre éditeur. Attention : jamais Word, qui ajoute des codes de mise en forme invisibles !
2. Taper le code de la structure minimale (doctype, html, head avec le charset et le title, puis body).
3. Enregistrer le fichier avec l'extension .html (par exemple index.html), en choisissant l'encodage UTF-8, dans le dossier du site.
4. Ouvrir le fichier dans un navigateur : double-clic sur le fichier, ou clic droit → « Ouvrir avec » → Chrome/Firefox.
5. Modifier puis actualiser : après chaque modification du code, enregistrer (Ctrl+S) puis appuyer sur la touche F5 dans le navigateur pour recharger la page et voir le résultat.

**EXPÉRIENCE** TP guidé : ma première page web

Objectif : afficher dans un navigateur une page présentant ton établissement.  
Matériel et logiciels : un ordinateur, le Bloc-notes (ou Notepad++), un navigateur.  
Manipulation :

1. Crée sur le Bureau un dossier nommé monsite.
2. Ouvre le Bloc-notes et recopie le code de l'exemple « Mon lycée » ci-dessus, en

adaptant le nom de l'établissement.

3. Enregistre sous `index.html` dans `monsite` (type de fichier : « Tous les fichiers », encodage : UTF-8).
4. Double-clique sur `index.html` : la page s'ouvre dans le navigateur.
5. Supprime volontairement la ligne `<meta charset="utf-8">`, enregistre et appuie sur F5.

Observations : sans le `charset`, les accents deviennent des symboles bizarres (Ã©, Ã"). Remets la ligne, actualise : tout redevient correct.

Interprétation : le navigateur interprète le fichier à chaque chargement ; l'encodage UTF-8 est indispensable pour le français.

#### EXERCICE RÉSOLU 1 Corriger une page mal formée

Énoncé. Un élève écrit le code suivant, mais sa page affiche « Ã©Ã"ves » et tout le texte est en gras. Trouve les deux erreurs et corrige-les.

```
<html>
<body>
  <h1>Résultats du probatoire
  <p><b>Liste des élèves admis</p>
</body>
</html>
```

#### SOLUTION

Solution détaillée. Erreur 1 : absence du `<head>` et surtout de `<meta charset="utf-8">`, d'où les accents corrompus. Erreur 2 : la balise `<b>` n'est jamais fermée, donc le gras se propage jusqu'à la fin de la page (la balise `<h1>` n'est pas fermée non plus). Code corrigé :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Résultats</title>
</head>
<body>
  <h1>Résultats du probatoire</h1>
  <p><b>Liste des élèves admis</b></p>
</body>
</html>
```

**EXERCICE 1 À toi de jouer**

Écris le code HTML complet d'une page intitulée « Mon quartier » contenant : un titre principal, une ligne horizontale, un sous-titre « Situation géographique », un paragraphe de deux lignes (avec `<br>`) comportant un mot en gras et un mot en italique, et un sous-titre « Écoles ». Vérifie l'affichage des accents dans le navigateur.

**CORRIGÉ Exercice 1**

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Mon quartier</title>
</head>
<body>
  <h1>Mon quartier</h1>
  <hr>
  <h2>Situation géographique</h2>
  <p>Mon quartier se trouve à <b>Nkongsamba</b>, <br>
  dans le département du <i>Moungo</i>.</p>
  <h2>Écoles</h2>
</body>
</html>
```

Points de vigilance : charset présent, un seul `<h1>`, balises `<b>` et `<i>` bien fermées, `<br>` orpheline sans fermeture.

**LESAVAIS-TU ? Le premier site web de l'histoire est toujours en ligne**

Le premier site web du monde a été créé en 1991 par l'informaticien britannique Tim Berners-Lee au CERN (laboratoire européen de physique, à Genève). Son objectif : permettre aux chercheurs de partager des documents reliés par des hyperliens. Cette page historique, qui explique ce qu'est le « World Wide Web », a été remise en ligne à son adresse d'origine : `info.cern.ch`. Tim Berners-Lee a refusé de breveter son invention, offrant le Web gratuitement à l'humanité : c'est grâce à ce choix qu'aujourd'hui, même un élève de Première TI à Nkongsamba peut publier sa propre page !

**À RETENIR L'essentiel de la section**

- Un site web est un ensemble de pages reliées par des hyperliens, hébergé sur un serveur et consulté via un navigateur ; un site statique a un contenu fixe.

- HTML est un langage à balises, interprété par le navigateur (jamais compilé).
- Une balise s'écrit `<nom>...</nom>`; les balises orphelines (`<br>`, `<hr>`, `<img>`) se ferment d'elles-mêmes; un attribut s'écrit `nom="valeur"` dans la balise ouvrante.
- Structure minimale : `<!DOCTYPE html>`, `<html>`, `<head>` (avec `<meta charset="utf-8">` et `<title>`), `<body>`.
- Sans `charset="utf-8"`, les accents s'affichent en caractères corrompus (Ã©).
- Balises de texte : `<h1>` à `<h6>`, `<p>`, `<br>`, `<hr>`, `<b>`, `<i>`.
- Cycle de travail : éditer → enregistrer en `.html` (UTF-8) → ouvrir dans le navigateur → actualiser avec F5.

## 2 LISTES, HYPERLIENS ET URL

Dans la section précédente, nous avons posé les fondations d'une page HTML : la structure globale (`<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`), les balises de titre (`<h1>` à `<h6>`), les paragraphes (`<p>`) et l'importance cruciale de la balise `<meta charset="utf-8">` pour afficher correctement nos accents camerounais (É, È, À, Ô, etc.). Une page web ne vit cependant pas isolée : elle doit structurer l'information sous forme de listes, et surtout, elle doit permettre à l'utilisateur de naviguer vers d'autres ressources. C'est précisément le rôle des listes et des hyperliens, que nous allons maîtriser maintenant.

### 2.1 Les listes en HTML : structurer l'information

Une page web présente souvent des ensembles d'éléments : un menu de navigation, les filières d'un lycée, les articles d'un panier d'achat, les étapes d'une recette de ndolé ou de poulet DG. HTML propose deux types principaux de listes, distinguées par la sémantique (le sens) que l'on veut donner à l'ordre des éléments.

#### DÉFINITION Liste non ordonnée et liste ordonnée

Une liste non ordonnée (`<ul>` pour Unordered List) regroupe des éléments dont l'ordre n'a pas d'importance sémantique (ex. : ingrédients d'une recette, liens d'un menu). Chaque élément est balisé `<li>` (List Item). Le navigateur affiche par défaut une puce (disque, cercle, carré) devant chaque `<li>`.

Une liste ordonnée (`<ol>` pour Ordered List) regroupe des éléments dont la séquence est significative (ex. : étapes d'un montage PC, classement des élèves, chapitres d'un cours). Chaque élément est aussi balisé `<li>`. Le navigateur affiche par défaut une numérotation (1, 2, 3... ou a, b, c... selon l'attribut `type`).

**PROPRIÉTÉ** Syntaxe de base des listes

- Non ordonnée : `<ul><li>Élément 1</li><li>Élément 2</li></ul>`
- Ordonnée : `<ol><li>Étape 1</li><li>Étape 2</li></ol>`
- Attributs utiles sur `<ol>` : `type="1|A|a|I|i"` (style de numérotation), `start="n"` (dé-marrer à n), `reversed` (ordre décroissant).
- Attribut `value="n"` sur un `<li>` spécifique pour forcer son numéro.

**EXEMPLE** Liste des filières du lycée technique de Bafoussam

Voici une liste non ordonnée pour des filières (l'ordre alphabétique est arbitraire) et une liste ordonnée pour la procédure d'inscription.

```
<!-- Liste non ordonnée : filières -->
<ul>
  <li>Génie Civil (GC)</li>
  <li>Électrotechnique (ET)</li>
  <li>Mécanique de Construction (MC)</li>
  <li>Informatique Industrielle (II)</li>
</ul>

<!-- Liste ordonnée : étapes d'inscription -->
<ol>
  <li>Se procurer le formulaire au secrétariat.</li>
  <li>Remplir les informations (nom, date de naissance, série).</li>
  <li>Joindre l'acte de naissance et le certificat de scolarité.</li>
  <li>Déposer le dossier complet avant la date limite.</li>
</ol>
```

**Imbrication des listes (listes imbriquées).**

C'est une compétence clé du programme : une liste peut contenir d'autres listes. Pour ce faire, on place une balise `<ul>` ou `<ol>` à l'intérieur d'un `<li>` parent, au même niveau que le texte de cet item. L'indentation du code (décalage vers la droite) devient alors indispensable pour la lisibilité et la maintenance.

**MÉTHODE** Construire une liste imbriquée proprement

1. Ouvrir la liste parente (`<ul>` ou `<ol>`).
2. Pour chaque item parent : ouvrir `<li>`, écrire le texte de l'item.
3. Si l'item a des sous-items : ouvrir la liste enfant (`<ul>` ou `<ol>`) avant de fermer le `</li>` parent.

4. Remplir la liste enfant avec ses propres `<li>`.
5. Fermer la liste enfant (`</ul>` ou `</ol>`).
6. Fermer le `</li>` parent.
7. Répéter pour les autres items parents.
8. Fermer la liste parente.
9. Indenter chaque niveau de 2 ou 4 espaces (ou une tabulation) pour visualiser la hiérarchie.

#### EXEMPLE Liste imbriquée : Filières → Séries → Matières principales

Cet exemple montre trois niveaux d'imbrication. Remarquez l'indentation rigoureuse : elle reflète exactement l'arborescence du DOM.

```
<ul>
  <li>Filière Scientifique
    <ul>
      <li>Série C (Mathématiques, Physique, Chimie)
        <ul>
          <li>Mathématiques (coeff. 6)</li>
          <li>Physique-Chimie (coeff. 5)</li>
          <li>SVT (coeff. 3)</li>
        </ul>
      </li>
      <li>Série D (Mathématiques, SVT, Physique-Chimie)
        <ul>
          <li>SVT (coeff. 6)</li>
          <li>Mathématiques (coeff. 5)</li>
          <li>Physique-Chimie (coeff. 4)</li>
        </ul>
      </li>
    </ul>
  </li>
  <li>Filière Littéraire
    <ul>
      <li>Série A4 (Espagnol, Allemand, Anglais)
        <ul>
          <li>Langues vivantes (coeff. 6)</li>
          <li>Philosophie (coeff. 4)</li>
          <li>Histoire-Géo (coeff. 3)</li>
        </ul>
      </li>
    </ul>
  </li>
  <li>Filière Technologique
    <ul>
      <li>Série F (Mécanique, Électrotechnique, Génie Civil)
        <ul>
          <li>Technologie de la spécialité (coeff. 7)</li>
          <li>Mathématiques (coeff. 4)</li>
        </ul>
      </li>
    </ul>
  </li>
</ul>
```

```
        <li>Physique-Chimie (coeff. 3)</li>
      </ul>
    </li>
  </ul>
</li>
</ul>
```

**ATTENTION** Piège classique : mauvaise fermeture des balises imbriquées

L'erreur n°1 chez les débutants est de fermer le `</li>` parent avant d'avoir ouvert la liste enfant. Le code suivant est invalide (mal formé) :

```
<!-- MAUVAIS : le </li> coupe la branche avant la sous-liste -->
<ul>
  <li>Filière Scientifique</li>
  <ul>
    <li>Série C</li>
  </ul>
</li>
</ul>
```

*Commentaires explicatifs :*

- `<!-- FERMETURE PRÉMATURÉE ! -->` : pointe sur `</li>` à la ligne 4.
- `<!-- Orpheline, hors d'un <li> -->` : pointe sur `<ul>` à la ligne 5.
- `<!-- </li> en trop, déséquilibre -->` : pointe sur `</li>` à la ligne 8.

Le validateur W3C ([validator.w3.org](https://validator.w3.org)) vous signalera une erreur de type « Element ul not allowed as child of element li in this context » ou « Unclosed element li ». Respectez toujours la règle : la liste enfant est contenue dans le `<li>` parent.

## 2.2 L'URL : adresse unique d'une ressource sur le Web

Avant de créer des liens, il faut comprendre où ils pointent. Chaque ressource sur le Web (page HTML, image, vidéo, fichier PDF, script, API) possède une adresse unique normalisée : l'URL (Uniform Resource Locator).

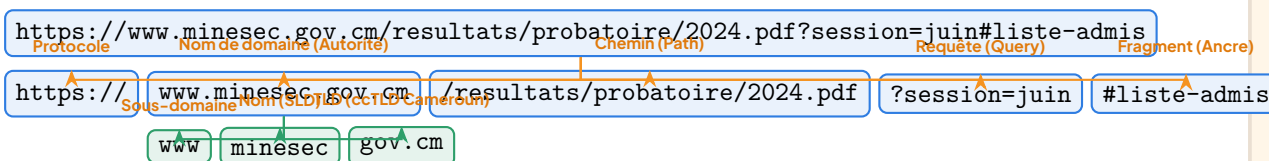
**DÉFINITION** URL (Uniform Resource Locator)

Une URL est une chaîne de caractères standardisée qui identifie l'emplacement d'une ressource sur Internet et le protocole pour y accéder. Elle se décompose en plusieurs parties obligatoires ou optionnelles :

- Le protocole (scheme) : `http://`, `https://`, `ftp://`, `mailto:`, `file://`... Indique comment communiquer.
- L'autorité (authority) : souvent un nom de domaine (ex. `www.minesec.gov.cm`) ou une adresse IP. Peut inclure un port (`:80`, `:443`) et des identifiants (`user:pass@` — déconseillé).
- Le chemin (path) : chemin hiérarchique vers la ressource sur le serveur (ex. `/resultats/probatoire/2024.pdf`). Sensible à la casse sur les serveurs Linux/Unix.

(majorité du Web).

- La requête (query string) : commence par ?, paires clé=valeur séparées par & (ex. ?annee=2024&session=juin).
- Le fragment (anchor) : commence par #, pointe vers une ancre id dans la page (ex. #inscription).



Décomposition anatomique d'une URL réelle du MINESUP/MINESEC camerounais. Noter le `https://` (sécurisé), le domaine `gov.cm` (administration camerounaise), le chemin hiérarchique, la requête et l'ancre finale.

#### PROPRIÉTÉ Règles de syntaxe d'une URL

- Les caractères non-ASCII (espaces, accents, é, à, ç) doivent être encodés en pourcent (percent-encoding) : espace → %20 ou +, é → %C3%A9.
- Les caractères réservés (: / ? # [ ] @ ! \$ & ' ( ) \* + , ; =) ont un sens spécial ; s'ils font partie d'une valeur de données, ils doivent être encodés.
- La longueur maximale pratique est d'environ 2000 caractères (limite navigateurs/-serveurs).
- HTTPS (port 443) est la norme actuelle : chiffre la communication (TLS/SSL). Tout site professionnel (banque, administration, école) doit l'utiliser. HTTP (port 80) transmet en clair (risque d'écoute).

## 2.3 Les hyperliens : la navigation avec <a>

L'hyperlien (ou simplement lien) est l'âme du Web. C'est lui qui transforme un ensemble de documents isolés en un hypertexte navigable.

#### DÉFINITION Hyperlien (balise <a>)

Un hyperlien est un élément interactif (texte, image, bloc) qui, au clic (ou activation clavier), demande au navigateur de charger la ressource désignée par son attribut `href` (Hypertext REFERENCE). La balise est `<a>` (ancre). Elle est inline par défaut (ne force pas

de saut de ligne).

- Attribut obligatoire : `href="URL"` (absolue ou relative).
- Attribut `title="Texte"` : infobulle (tooltip) affichée au survol souris ; utile pour l'accessibilité et l'UX.
- Attribut `target="_blank"` : ouvre dans un nouvel onglet/fenêtre. Obligatoire d'ajouter `rel="noopener noreferrer"` pour la sécurité (empêche la page ouverte d'accéder à `window.opener`).
- Attribut `download` : force le téléchargement au lieu de l'affichage (pour PDF, ZIP, images).

### Lien absolu vs lien relatif.

C'est une distinction fondamentale pour l'organisation d'un site (dossiers locaux vs ressources externes).

#### PROPRIÉTÉ Lien absolu

Contient l'URL complète (protocole + domaine + chemin). Pointe vers n'importe quelle ressource sur le Web, y compris sur le même domaine mais de façon explicite.

```
<a href="https://www.minesec.gov.cm/examens/probatoire">Résultats Probatoire MINESEC</a>  
<a href="https://www.camtelecom.cm/fibre-optique">CAMTEL Fibre Optique</a>
```

#### PROPRIÉTÉ Lien relatif

Ne contient que le chemin relatif à la page courante. Ne fonctionne qu'à l'intérieur du même site (même domaine). Indispensable pour la portabilité : si vous déplacez le site sur un autre domaine (ex. de `localhost` vers `monlycee.cm`), les liens relatifs fonctionnent sans modification.

- `href="page.html"` : même dossier.
- `href="dossier/page.html"` : sous-dossier (descendre).
- `href="../page.html"` : dossier parent (remonter d'un niveau). `..` = parent.
- `href="../../autre/page.html"` : remonter de deux niveaux puis descendre.
- `href="/"` ou `href="/index.html"` : relatif à la racine du site (démontre par `/`). Très robuste.

**MÉTHODE** Choisir entre lien absolu et lien relatif

1. La ressource est-elle sur un autre domaine (site externe)?
  - OUI → Lien absolu obligatoire (ex. lien vers Wikipedia, CAMTEL, Facebook, Google Fonts).
2. La ressource est-elle sur le même site (même domaine)?
  - OUI → Privilégier le lien relatif.
  - Si la structure de dossiers est stable et simple : relatif au fichier courant (page.html, dossier/page.html, ../page.html).
  - Si la structure peut changer ou pour les liens de navigation globale (menu, logo, pied de page) → relatif à la racine (/index.html, /contact.html, /assets/css/style.css). C'est la meilleure pratique professionnelle.
3. Testez toujours en cliquant sur le lien dans le navigateur (local file:// ou serveur http://localhost). Vérifiez la barre d'adresse et la console (F12) pour les erreurs 404.

**Lien vers une ancre interne (fragment d'URL).**

Permet de sauter à une section précise dans la même page ou dans une autre page. On utilise l'attribut universel `id` sur l'élément cible.

**EXEMPLE** Navigation interne avec ancres (sommaire de page)

```
<!-- Dans la même page : sommaire en haut -->
<nav>
  <ul>
    <li><a href="#presentation">Présentation du lycée</a></li>
    <li><a href="#filieres">Nos filières</a></li>
    <li><a href="#inscription">Inscription 2024-2025</a></li>
    <li><a href="#contact">Contact \& Accès</a></li>
  </ul>
</nav>

<!-- Plus bas dans la page : sections cibles -->
<section id="presentation">
  <h2>Présentation du Lycée Technique de Douala</h2>
  <p>...</p>
</section>

<section id="filieres">
  <h2>Filières proposées</h2>
  <!-- Liste imbriquée vue plus haut -->
</section>

<section id="inscription">
  <h2>Modalités d'inscription</h2>
  <ol>...</ol>
```

```

</section>

<footer id="contact">
  <address>BP 1234 Douala – Tél : +237 2XX XX XX XX</address>
</footer>

```

#### PROPRIÉTÉ Lien vers ancre dans une autre page

Combine chemin relatif/absolu + #id.

```

<!-- Depuis index.html vers la section "inscription" de infos.html -->
<a href="infos.html#inscription">Voir les modalités d'inscription</a>

<!-- Lien absolu vers ancre externe -->
<a href="https://www.minesec.gov.cm/resultats#probatoire">Résultats Probatoire sur le site MINESEC</a>

```

**Lien mailto :** : ouvrir le client de messagerie.

Utile pour les pages « Contact ». Ouvre le logiciel de messagerie par défaut (Outlook, Thunderbird, Gmail web, Mail iOS/Android) avec des champs pré-remplis.

#### PROPRIÉTÉ Syntaxe mailto:

```

<!-- Simple -->
<a href="mailto:secretariat@lyceebilingue.cm">Écrire au secrétariat</a>

<!-- Avec sujet, corps, copies (encodage URL requis pour espaces et accents) -->
<a href="mailto:direction@lyceebilingue.cm?subject=Demande%20de%20renseignements%20inscription&body=Bonjour%2C%20DAJe%20souhaite%20inscrire%20mon%20enfant%20en%201%C3%A8re%20Tl.&cc=comptabilite@lyceebilingue.cm">
  Contacter la direction (pré-rempli)
</a>

```

**Attention :** mailto : dépend de la configuration poste client (pas de client mail → ne fait rien ou propose webmail). Ne pas l'utiliser pour des actions critiques serveur (utilisez un formulaire <form action="..."> traité par un script côté serveur — en Terminale).

#### EXEMPLE Menu de navigation complet d'un site de lycée (3 liens + infobulles)

Ce menu utilise des liens relatifs à la racine (/ . . .) pour la robustesse, des infobulles title explicites, et une structure sémantique <nav><ul><li><a>.

```
<nav aria-label="Navigation principale du site">
  <ul class="menu-principal">
    <li>
      <a href="/index.html"
        title="Page d'accueil du Lycée Technique de Bafoussam – Actualités et informations
générales">
        Accueil
      </a>
    </li>
    <li>
      <a href="/filières.html"
        title="Découvrir nos 5 filières techniques : GC, ET, MC, II, Froid/Climatisation">
        Filières
      </a>
    </li>
    <li>
      <a href="/inscription.html"
        title="Procédure d'inscription, frais de scolarité, calendrier 2024-2025">
        Inscription
      </a>
    </li>
    <!-- Exemple de lien externe absolu avec target="_blank" sécurisé -->
    <li>
      <a href="https://www.minesec.gov.cm"
        target="_blank"
        rel="noopener noreferrer"
        title="Site officiel du Ministère des Enseignements Secondaires (s'ouvre dans un nouvel
onglet)">
        MINESEC
      </a>
    </li>
  </ul>
</nav>
```

**ATTENTION** Erreur n°1 : le lien relatif cassé après déplacement de fichier

C'est la cause numéro un de liens brisés (erreur 404) en développement local.

Scénario : Vous créez `index.html` et `apropos.html` à la racine. Dans `index.html`, vous mettez `<a href="apropos.html">À propos</a>`. Ça marche. Puis vous créez un dossier `pages/` et vous y déplacez `apropos.html` pour ranger. Vous oubliez de mettre à jour le lien dans `index.html`. Le navigateur cherche `apropos.html` à la racine → 404 Not Found.

**MÉTHODE** Éviter les liens relatifs cassés

1. Planifiez l'arborescence avant de coder (dossiers `css/`, `js/`, `images/`, `pages/`).
2. Utilisez des liens relatifs à la racine (`/pages/apropos.html`) pour la navigation principale. Ils ne cassent jamais quand on déplace le fichier source du lien.
3. Utilisez un éditeur avec renommage/refactoring (VS Code : clic droit → Rename Symbol / Move File met à jour les imports/liens automatiquement dans tout le projet).
4. Testez systématiquement après tout déplacement de fichier (raccourci Ctrl+Click sur le `href` dans VS Code pour sauter à la cible).
5. Vérifiez la console réseau (F12 → Network) : les requêtes en rouge (statut 404) révèlent les ressources introuvables (CSS, JS, images, liens).

/ (racine site)

pages/

assets/



Arborescence simplifiée d'un site de lycée et liens relatifs entre `index.html`, `pages/apropos.html` et `assets/style.css`. Noter l'usage de `../` pour remonter depuis `pages/` vers la racine, et `/assets/...` (relatif à la racine) qui fonctionne depuis n'importe où.

## 2.4 Bonnes pratiques, accessibilité et limites du HTML statique

Au-delà de la syntaxe, un développeur web professionnel (même en Première TI) doit adopter des réflexes de qualité.

**À RETENIR** Check-list « Liens de qualité » pour le Probatoire et au-delà

- Texte de lien explicite : Évitez « Cliquez ici », « Lien », « Ici ». Préférez « Consulter les résultats du Probatoire 2024 », « Télécharger le programme de 1ère TI (PDF) ». Le texte seul doit indiquer la destination (accessibilité lecteurs d'écran, SEO, UX).
- Infobulle title : Ajoutez une information supplémentaire (pas une répétition du texte). Ex. : `<a href="/inscription" title="Frais : 15 000 FCFA - Date limite : 30 sept.">Inscription</a>`.
- Liens externes : `target="_blank" rel="noopener noreferrer"` systématique. Indiquez visuellement (icône `<span aria-hidden="true">☐↑</span>` ou texte « (nouvel onglet) ») que le lien sort du site.
- Encodage des URLs : Dans href, les espaces deviennent %20, les accents s'encodent (UTF-8 → %XX). En HTML5, le navigateur gère souvent l'affichage, mais le code source doit être valide.
- Validation : Passez vos pages au validateur W3C (validator.w3.org) : « Validate by direct input » ou « Validate by file upload ». Corrigez toutes les erreurs (balises non fermées, attributs invalides, `<a>` sans href, id dupliqués).

**LESAVAIS-TU ?** Le premier lien de l'histoire du Web

Le 6 août 1991, Tim Berners-Lee a mis en ligne la première page web au CERN (Suisse). Elle expliquait le projet « WorldWideWeb » et contenait... des hyperliens vers d'autres documents du CERN. L'URL était `http://info.cern.ch/hypertext/WWW/TheProject.html`. Elle existe toujours (archivée). Au Cameroun, la première connexion Internet opérationnelle date de 1996 (via le projet NICI et CAMTEL). Aujourd'hui, la fibre optique nationale (backbone CAMTEL, projet Patriot) relie les 10 régions, permettant aux lycées techniques d'accéder aux ressources pédagogiques en ligne et de publier leurs propres sites d'établissement.

**EXERCICE RÉSOLU 2** Corriger des liens relatifs cassés

Énoncé. L'arborescence d'un site de gestion de notes est la suivante :

```

/ (racine)===
index.html===
css/===
  === style.css===
pages/===
  === eleves.html===
  === notes/===
    === saisie.html===
  
```

```
assets/  
≡≡≡ logo.png
```

Dans `pages/notes/saisie.html`, on trouve ces liens (certains sont erronés) :

1. `<a href="../../index.html">Accueil</a>`
2. `<a href="eleves.html">Liste élèves</a>`
3. `<a href="/css/style.css">Feuille de style</a>` (dans une balise `<link>`)
4. `<a href="../../assets/logo.png">Logo</a>` (dans une balise `<img>`)

Identifiez les erreurs et corrigez-les.

**Solution détaillée.** Analysons la position de `saisie.html` : `/pages/notes/saisie.html`. Profondeur : 3 niveaux (pages → notes → saisie.html).

1. `href="../../index.html"` : `../` remonte à `/pages/`, second `../` remonte à `/` (racine). **Correct.** Atteint `/index.html`.
2. `href="eleves.html"` : chemin relatif sans préfixe, donc cherche dans le même dossier que la page courante, soit `/pages/notes/eleves.html`. **ERREUR** : `eleves.html` est dans `/pages/` (un niveau au-dessus).
  - **Correction** : `href="../eleves.html"` (remonter d'un niveau vers `/pages/`) OU `href="/pages/eleves.html"` (relatif à la racine, recommandé).
3. `href="/css/style.css"` (dans `<link rel="stylesheet" href="...">`) : commence par `/`, donc relatif à la racine. Chemin résolu : `/css/style.css`. **Correct.** C'est la meilleure pratique pour les ressources partagées (CSS, JS, images communes).
4. `href="../../assets/logo.png"` (dans ``) : `../` remonte à `/pages/`. Cherche `/pages/assets/logo.png`. **ERREUR** : le dossier `assets/` est à la racine, pas dans `/pages/`.
  - **Correction** : `href="../../assets/logo.png"` (remonter deux niveaux : notes → pages → racine) OU `href="/assets/logo.png"` (relatif à la racine, fortement recommandé).

**Règle d'or** : Pour les ressources globales (CSS, JS, logo, favicon, polices), utilisez toujours le chemin relatif à la racine (`/...`). Cela évite tout calcul de `../` selon la profondeur de la page courante.

## EXERCICE 2 Entraînement : listes, liens et URLs

1. Écrivez le code HTML d'une liste ordonnée présentant les 4 étapes du montage d'un processeur Intel LGA1700 sur carte mère (ouverture levier, alignement triangle, pose CPU, fermeture levier/verrouillage). Utilisez l'attribut `type="I"` pour une numérotation en chiffres romains majuscules.
2. Créez une liste non ordonnée imbriquée sur 2 niveaux pour un menu « Services » d'un

cybercafé à Yaoundé : « Accès Internet » (sous-items : « Horaire 1h », « Forfait journée », « Fibre 100 Mbps »), « Impression/Scan » (sous-items : « N&B A4 », « Couleur A4 », « Scan vers email »), « Formation » (sous-items : « Initiation bureautique », « Maintenance PC », « Code HTML/CSS »).

3. Donnez l'URL absolue complète pour accéder à la page « Résultats BAC 2024 » du site `www.officebac.cm` en HTTPS, avec un paramètre de requête `session=juin` et une ancre `#serie-TI`. Décomposez-la (protocole, domaine, chemin, requête, fragment).
4. Dans une page `/pages/contact.html`, écrivez le code du lien vers `/index.html` (relatif à la racine), le lien vers `/pages/apropos.html` (relatif à la racine), et le lien `mailto:` pré-rempli vers `admin@lycee.cm` avec sujet « Contact site web » et corps « Bonjour, je souhaite... ».
5. Expliquez pourquoi le lien `<a href="page.html">Lien</a>` dans `/dossier/sous/page.html` ne pointe pas vers `/page.html` mais vers `/dossier/sous/page.html`. Comment corriger pour viser la racine ?

#### **CORRIGÉ** Exercice n°5 – Listes, liens et URLs

```
<ol type="I">
  <li>Ouvrir le levier de rétention du socket LGA1700 (relever la tige métallique).</li>
  <li>Aligner le triangle doré du processeur avec le triangle du socket (coins correspondants).</li>
  <li>Poser délicatement le processeur à la verticale sans le glisser (pas de force).</li>
  <li>Refermer le levier de rétention et verrouiller la tige sous l'ergot de fixation.</li>
</ol>
```

```
<ul>
  <li>Accès Internet
    <ul>
      <li>Horaire 1h</li>
      <li>Forfait journée</li>
      <li>Fibre 100 Mbps</li>
    </ul>
  </li>
  <li>Impression/Scan
    <ul>
      <li>N&B A4</li>
      <li>Couleur A4</li>
      <li>Scan vers email</li>
    </ul>
  </li>
  <li>Formation
    <ul>
      <li>Initiation bureautique</li>
      <li>Maintenance PC</li>
      <li>Code HTML/CSS</li>
    </ul>
  </li>
</ul>
```

```
</li>
</ul>
```

3. URL : `https://www.officebac.cm/resultats/bac-2024?session=juin#serie-TI`

- Protocole : `https://`
- Domaine : `www.officebac.cm`
- Chemin : `/resultats/bac-2024`
- Requête : `?session=juin`
- Fragment : `#serie-TI`

```
<!-- Dans /pages/contact.html -->
<nav>
  <ul>
    <li><a href="/index.html" title="Retour à la page d'accueil du site">Accueil</a></li>
    <li><a href="/pages/apropos.html" title="À propos de notre établissement">À propos</a></li>
    <li><a href="mailto:admin@lycee.cm?subject=Contact%20site%20web&body=Bonjour%20C%0D%0AJe%20souhaite..." title="Envoyer un email à l'administrateur">Nous contacter</a></li>
  </ul>
</nav>
```

5. Explication : Un lien relatif sans slash initial (`page.html`) est résolu par rapport au dossier de la page courante. Ici, page courante = `/dossier/sous/page.html`, son dossier = `/dossier/sous/`. Le navigateur construit `/dossier/sous/page.html`. Correction : Utiliser le chemin relatif à la racine : `href="/page.html"` (le slash initial / signifie « depuis la racine du site »). Ainsi, peu importe où se trouve le fichier source, le lien pointe toujours vers `/page.html` à la racine.

Vous maîtrisez maintenant les listes (simples et imbriquées), l'anatomie d'une URL, et la création de liens hypertextes robustes (absolus, relatifs, ancrés, mailto) avec leurs attributs d'accessibilité (`title`, `rel`). La prochaine section abordera l'insertion des **médias** (images, sons, vidéos) pour enrichir visuellement vos pages, avant de passer aux tableaux et formulaires.

### 3 INSERTION DE MÉDIAS : IMAGES, SONS, VIDÉOS

Dans la section précédente, tu as appris à structurer le texte d'une page avec les listes et à relier les pages entre elles grâce aux hyperliens et aux URL. Mais un site composé uniquement de texte ressemblerait à un vieux document tapé à la machine ! Imagine le site web du lycée sans le logo de l'établissement, sans la photo de la cérémonie de remise des prix, sans l'hymne national en fond sonore : il serait terne et peu attractif. Les **médias** (images, sons, vidéos) transforment une simple page en une véritable vitrine. C'est exactement ce que fait un cybercafé de Douala qui crée le site d'une agence de tourisme : il y insère des photos du Mont Cameroun, une

vidéo des chutes de la Lobé et des chants traditionnels. Dans cette section, tu vas apprendre à insérer chacun de ces médias avec la syntaxe HTML exacte, à choisir le bon chemin de fichier, et à éviter les pièges classiques.

### 3.1 Insertion d'une image : la balise `<img>`

**Intuition.** Une page HTML ne « contient » pas réellement les images : elle contient seulement une référence vers le fichier image, comme une affiche publicitaire indique où trouver un produit. Le navigateur lit cette référence, va chercher le fichier image (sur le même ordinateur ou sur un serveur distant), puis l'affiche à l'endroit prévu.

#### DÉFINITION Balise `<img>`

La balise `<img>` (de l'anglais image) permet d'insérer une image dans une page web. C'est une **balise orpheline** (ou balise vide) : elle ne possède pas de balise fermante. Sa syntaxe générale en HTML5 est :

```

```

#### PROPRIÉTÉ Attributs de la balise `<img>`

- **src** (source) : obligatoire. Indique l'URL de l'image, c'est-à-dire l'adresse du fichier image à afficher.
- **alt** (alternative) : fortement recommandé (obligatoire pour la validité HTML5). Texte alternatif affiché si l'image ne peut pas être chargée, et lu à voix haute par les logiciels d'accessibilité.
- **width** et **height** : largeur et hauteur d'affichage, exprimées en **pixels** (valeurs numériques entières, sans unité **px** dans l'attribut). Si l'on n'indique qu'une seule dimension, le navigateur calcule l'autre en conservant les proportions.

#### ATTENTION `<img>` est une balise orpheline

Contrairement à `<p>...</p>` ou `<h1>...</h1>`, la balise `<img>` ne se ferme jamais avec `</img>`. Écrire `</img>` est une erreur fréquente au Probatoire. Toute l'information se trouve dans les attributs, à l'intérieur de la balise unique :

```

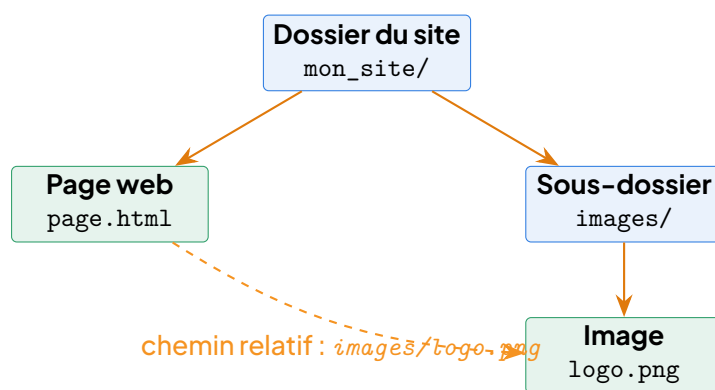
```

Même remarque pour `<br>`, `<hr>`, `<meta>`, `<input>` : ce sont aussi des balises orphelines.

### 3.2 Image locale (chemin relatif) ou image distante (URL absolue) ?

L'attribut `src` accepte deux types d'adresses, que tu connais déjà depuis la section sur les hyperliens :

- **Image locale (chemin relatif)** : le fichier image est rangé dans les dossiers du site, par exemple dans un sous-dossier `images/`. On écrit alors `src="images/logo.png"`. C'est la méthode recommandée pour un site sérieux : l'image voyage avec le site.
- **Image distante (URL absolue)** : l'image est hébergée sur un autre serveur. On écrit alors l'adresse complète : `src="https://www.exemple.cm/photo.jpg"`. Pratique pour tester, mais risqué : si le site distant supprime l'image, elle disparaît de ta page (lien brisé)!



Chemin relatif d'une image : depuis `page.html`, on descend dans le dossier `images/` pour atteindre `logo.png`.

#### MÉTHODE

#### Insérer une image avec le bon chemin relatif

Pour insérer proprement une image locale dans ta page, suis ces étapes :

1. Dans le dossier de ton site (par exemple `mon_site/`), crée un sous-dossier nommé `images`.
2. Copie ton fichier image (par exemple `logo.png`) dans ce sous-dossier.
3. Vérifie l'orthographe exacte du nom de fichier : `logo.png` et `Logo.PNG` sont deux noms différents pour de nombreux serveurs (respecte les majuscules et l'extension).
4. Dans ta page HTML, écris la balise avec le chemin relatif : ``.
5. Ouvre la page dans le navigateur : si l'image s'affiche, le chemin est correct ; sinon, vérifie le nom du dossier, le nom du fichier et l'extension.

### 3.3 Le texte alternatif `alt` : bien plus qu'une option

#### DÉFINITION Texte alternatif

Le **texte alternatif** (attribut `alt`) est une courte description de l'image. Il remplit deux rôles essentiels :

- il s'affiche à la place de l'image si celle-ci ne peut pas être chargée (fichier supprimé, chemin erroné, connexion lente) ;
- il est lu à voix haute par les lecteurs d'écran utilisés par les personnes malvoyantes, rendant la page accessible à tous.

#### LESAVAIS-TU ? L'attribut `alt` et l'accessibilité

Les personnes aveugles ou malvoyantes naviguent sur le web grâce à des logiciels appelés **lecteurs d'écran**, qui lisent à voix haute le contenu des pages. Quand le logiciel rencontre une image, il lit le contenu de l'attribut `alt`. Une image sans `alt` est donc « invisible » pour ces utilisateurs. C'est pourquoi les normes internationales du web (W3C) exigent que toute image informative possède un texte alternatif. Rédiger un bon `alt`, c'est rendre ton site ouvert à tous, y compris aux élèves malvoyants de ton lycée.

### 3.4 L'image cliquable

**Intuition.** Sur de nombreux sites, cliquer sur le logo en haut de la page ramène à l'accueil. Comment faire ? Il suffit de combiner deux balises que tu connais déjà : le lien `<a>` et l'image `<img>`.

#### PROPRIÉTÉ Image cliquable

Pour rendre une image cliquable, on imbrique la balise `<img>` à l'intérieur d'une balise de lien `<a href="...">...</a>`. L'image remplace alors le texte du lien :

```
<a href="destination.html"></a>
```

#### EXEMPLE Logo du lycée cliquable renvoyant vers l'accueil

Sur le site du Lycée de Nkol-Eton, le logo affiché en haut de chaque page renvoie vers la page d'accueil `index.html` :

```
<!-- Logo cliquable : un clic ramene à la page d'accueil -->
<a href="index.html" title="Retour à l'accueil">
  
</a>
```

L'attribut `title` du lien ajoute une **infobulle** : le texte « Retour à l'accueil » apparaît quand la souris survole le logo.

**ATTENTION** Ne pas confondre `src` et `href`

C'est l'une des erreurs les plus fréquentes :

- `src` (source) indique où se trouve le média à afficher : il s'utilise avec `<img>`, `<audio>`, `<video>`, `<source>`.
- `href` (hypertext reference) indique la destination d'un lien : il s'utilise avec `<a>`, `<link>`.

Écrire `<img href="...">` ou `<a src="...">` ne fonctionne pas. Retiens : `src` = Source du média ; `href` = Hyperlien.

### 3.5 Insertion d'un son : la balise `<audio>`

**Intuition.** Un site culturel présentant les danses traditionnelles du Cameroun gagne à proposer un extrait musical. Avant HTML5, il fallait des logiciels externes (plugins) ; aujourd'hui, une seule balise suffit.

**DÉFINITION** Balise `<audio>`

La balise `<audio>` insère un lecteur sonore dans la page. L'attribut `src` indique l'URL du fichier audio, et l'attribut `controls` (booléen, sans valeur) fait apparaître les boutons de lecture (lecture, pause, volume) :

```
<audio src="sons/hymne.mp3" controls></audio>
```

Si le navigateur ne supporte pas le format, le contenu placé entre les balises (texte de secours) s'affiche.

**ATTENTION** Toujours mettre `controls`

Sans l'attribut `controls`, le lecteur audio est invisible : le son ne peut pas être déclenché par le visiteur (sauf lecture automatique déconseillée). Sauf cas particulier, ajoute toujours `controls`. Le format audio le plus courant et le mieux supporté est le **MP3**.

### 3.6 Insertion d'une vidéo : la balise `<video>`

**DÉFINITION** Balise <video>

La balise <video> insère un lecteur vidéo dans la page. On utilise `src` pour l'URL du fichier, `controls` pour afficher les commandes de lecture, `width` (éventuellement `height`) pour régler la taille du lecteur en pixels, et `poster` pour une image d'aperçu avant lecture :

```
<video src="videos/chutes_lobe.mp4" controls width="480" poster="images/apercu.jpg"></video>
```

Le format vidéo le plus répandu sur le web est le **MP4** (codec H.264).

### 3.7 Formats de fichiers et poids des médias

**À RETENIR** Formats courants à connaître

| Média | Formats courants    | Remarque                                                                                    |
|-------|---------------------|---------------------------------------------------------------------------------------------|
| Image | JPG, PNG, GIF, WebP | JPG : photos ; PNG : logos avec transparence ; GIF : animations ; WebP : moderne, compressé |
| Son   | MP3, OGG, WAV       | MP3 : standard universel ; OGG : alternative libre                                          |
| Vidéo | MP4, WebM, OGV      | MP4 (H.264) : standard universel ; WebM : alternative libre                                 |

**Poids des fichiers et temps de chargement.** Au Cameroun, de nombreux visiteurs naviguent avec une connexion mobile limitée. Une image de 5 Mo peut mettre plus d'une minute à s'afficher et décourager le visiteur. Il faut donc **optimiser** les médias : redimensionner les photos à la taille réellement affichée (inutile d'envoyer une photo de 4000 pixels de large pour l'afficher en 400), choisir le format adapté (JPG pour les photos, PNG pour les logos) et compresser les fichiers. Une image de page web devrait idéalement peser moins de 200 Ko.

**EXERCICE RÉSOLU 3** Page « Tourisme au Cameroun »

Énoncé. On te demande de créer la page d'accueil d'un site « Tourisme au Cameroun » contenant : le titre de la page, une photo du Mont Cameroun (fichier `mont.jpg` rangé dans `images/`, affichée en 400 px de large), un logo cliquable (`images/logo.png`) renvoyant vers `index.html`, et une vidéo des chutes de la Lobé (`videos/lobe.mp4`, largeur 480 px).

**SOLUTION**

Solution détaillée. On veille à : déclarer le jeu de caractères `utf-8` pour les accents, utiliser

des chemins relatifs corrects, fournir un `alt` à chaque image, et ajouter `controls` à la vidéo.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Tourisme au Cameroun</title>
</head>
<body>
  <h1>Bienvenue au Cameroun, l'Afrique en miniature</h1>

  <!-- Logo cliquable : retour à l'accueil -->
  <a href="index.html" title="Retour à l'accueil">
    
  </a>

  <h2>Le Mont Cameroun</h2>
  

  <h2>Les chutes de la Lobe</h2>
  <video src="videos/lobe.mp4" controls width="480"></video>
</body>
</html>
```

### Tourisme au Cameroun — Navigateur

H1 : Bienvenue au Cameroun, l'Afrique en miniature

#### Image cliquable

```
<a href="index.html">

```

un clic renvoie vers l'accueil

#### Image

```

```

#### Vidéo

```
<video src="videos/lobe.mp4" controls width="480">
  boutons lecture / pause / volume
```

Maquette de la page « Tourisme au Cameroun » : logo cliquable, image avec texte alternatif et lecteur vidéo.

**EXPÉRIENCE** TP guidé : insérer tes premiers médias

Objectif : créer une page affichant une image, une image cliquable et un son. Matériel : un ordinateur, le Bloc-notes ou Notepad++, un navigateur, une image `photo.jpg` et un fichier `chant.mp3`.

1. Crée le dossier `mon_site`, puis les sous-dossiers `images` et `sons`.
2. Place `photo.jpg` dans `images` et `chant.mp3` dans `sons`.
3. Dans `mon_site`, crée le fichier `medias.html` avec le code suivant :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Mes medias</title>
</head>
<body>
  <h1>Mes premiers medias</h1>
  <a href="medias.html" title="Recharger la page">
    
  </a>
  <audio src="sons/chant.mp3" controls></audio>
</body>
</html>
```

4. Ouvre `medias.html` dans le navigateur. Observations : l'image s'affiche, cliquer dessus recharge la page, le lecteur audio propose lecture/pause.
5. Renomme volontairement le fichier image, puis recharge : le texte de l'attribut `alt` apparaît à la place. Interprétation : le chemin relatif est devenu faux, et `alt` joue son rôle de secours.

**EXERCICE 3** À toi de jouer

Crée une page `culture.html` présentant une région du Cameroun : une image locale avec texte alternatif et largeur 350 px, un extrait sonore MP3, une vidéo MP4 de largeur 500 px, et une image cliquable renvoyant vers `index.html`. Soigne l'organisation des dossiers (`images/`, `sons/`, `videos/`).

**CORRIGÉ** Exercice

```
<!DOCTYPE html>
<html>
<head>
```

```
<meta charset="utf-8">
<title>Culture du Cameroun</title>
</head>
<body>
  <h1>La region de l'Ouest</h1>
  <a href="index.html" title="Retour à l'accueil">
    
  </a>
  
  <audio src="sons/rythme.mp3" controls></audio>
  <video src="videos/danse.mp4" controls width="500"></video>
</body>
</html>
```

Points de vigilance : balise `<img>` orpheline, `src` pour les médias et `href` pour le lien, `controls` présent sur le son et la vidéo, chemins relatifs cohérents avec l'organisation des dossiers.

#### À RETENIR L'essentiel de la section

- `` insère une image ; c'est une balise orpheline.
- Le chemin peut être relatif (fichier local, recommandé) ou absolu (URL distante).
- `alt` sert à l'accessibilité et s'affiche si l'image est manquante.
- Image cliquable : `<img>` imbriquée dans `<a href="...">`.
- `<audio src="..." controls>` pour un son (MP3), `<video src="..." controls width="...">` pour une vidéo (MP4).
- `src` = source du média ; `href` = destination d'un lien.
- Optimise le poids des fichiers pour accélérer le chargement des pages.

## 4 LES TABLEAUX EN HTML

Dans la section précédente, tu as appris à insérer des médias (images, sons, vidéos) pour rendre tes pages vivantes. Mais une page web ne contient pas que du texte et des images : il arrive très souvent qu'on doive présenter des **données organisées en lignes et en colonnes**. Pense à l'emploi du temps affiché au tableau du lycée, au bulletin de notes que le proviseur fait imprimer en fin de trimestre, ou au tableau des prix dans une boutique de Douala. Comment représenter proprement ces grilles d'informations dans une page web ? C'est exactement le rôle des **tableaux HTML**.

## 4.1 Structure d'un tableau : <table>, <tr>, <td>, <th>

### DÉFINITION Tableau HTML

Un **tableau** HTML est un élément de page web qui organise des données en lignes et en colonnes, formant des cellules à leur intersection. Il se construit avec quatre balises essentielles :

- `<table>` `</table>` : délimite le tableau complet ;
- `<tr>` `</tr>` (table row) : délimite une ligne du tableau ;
- `<td>` `</td>` (table data) : délimite une cellule ordinaire contenant des données ;
- `<th>` `</th>` (table header) : délimite une cellule d'en-tête (affichée en gras et centrée par défaut).

L'idée directrice est simple : un tableau se décrit **ligne par ligne**, et chaque ligne se décrit **cellule par cellule**, de gauche à droite. On n'écrit jamais les colonnes directement : ce sont les cellules des différentes lignes qui, empilées, forment les colonnes.

### PROPRIÉTÉ Règles d'imbrication

1. Toute cellule (`<td>` ou `<th>`) doit se trouver à l'intérieur d'une ligne `<tr>`.
2. Toute ligne `<tr>` doit se trouver à l'intérieur de `<table>`.
3. Le nombre de colonnes du tableau est déterminé par la ligne qui contient le plus de cellules.

### EXEMPLE Premier tableau : les notes de deux élèves

```
<table border="1">
  <tr>
    <th>Nom</th>
    <th>Maths</th>
    <th>Informatique</th>
  </tr>
  <tr>
    <td>Etoundi</td>
    <td>14</td>
    <td>17</td>
  </tr>
  <tr>
    <td>Kamga</td>
    <td>11</td>
    <td>15</td>
  </tr>
```

```
</table>
```

Ce code produit un tableau de 3 lignes et 3 colonnes : la première ligne est une ligne d'entêtes (<th>), les deux suivantes contiennent les données (<td>).

**ATTENTION** Balise fermante oubliée

L'erreur la plus fréquente du débutant est d'oublier un </td> ou un </tr>. Le navigateur tente alors de « deviner » la structure, et le tableau s'affiche déformé ou décalé. Indente ton code comme dans l'exemple : chaque niveau d'imbrication décalé vers la droite rend les oublis visibles immédiatement.

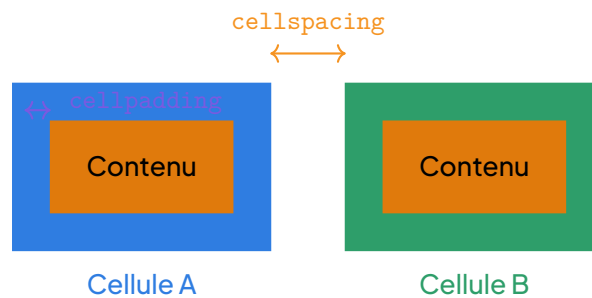
## 4.2 Les attributs du tableau : border, id, width, height, align, cellpadding, cellspacing

Sans attribut, un tableau s'affiche **sans bordures** et avec une taille automatique. Les attributs de la balise <table> permettent de contrôler son apparence.

| Attribut        | Rôle                                                                                                                    |
|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| border="1"      | Affiche une bordure d'épaisseur 1 pixel autour du tableau et des cellules.<br>border="0" (ou absence) : pas de bordure. |
| id="notes"      | Donne un identifiant unique au tableau (utile pour le CSS et le JavaScript).                                            |
| width="80%"     | Largeur du tableau : en pixels (width="500") ou en pourcentage de la page.                                              |
| height="200"    | Hauteur du tableau en pixels ou en pourcentage.                                                                         |
| align="center"  | Positionne le tableau dans la page : left, center ou right.                                                             |
| cellpadding="5" | Marge <b>intérieure</b> de chaque cellule : espace entre le bord de la cellule et son contenu (en pixels).              |
| cellspacing="3" | Espace <b>entre</b> les cellules voisines (en pixels).                                                                  |

**DÉFINITION** cellpadding **et** cellspacing

- **cellpadding** : espace à l'intérieur de chaque cellule, entre sa bordure et le texte qu'elle contient. C'est le « rembourrage » de la cellule.
- **cellspacing** : espace entre les cellules, c'est-à-dire l'épaisseur du vide qui sépare deux cellules voisines.

**EXEMPLE** Tableau avec attributs

```
<table border="1" id="resultats" width="70%" align="center"
  cellpadding="8" cellspacing="2">
  <tr><th>Matiere</th><th>Note</th></tr>
  <tr><td>Informatique</td><td>16</td></tr>
</table>
```

Le tableau occupe 70 % de la largeur de la page, est centré, et chaque cellule possède 8 pixels de marge intérieure.

**ATTENTION** Attributs historiques mais exigibles

`border`, `align`, `width`, `cellpadding` et `cellspacing` sont des attributs HTML historiques : dans la pratique professionnelle actuelle, on préfère les définir en CSS. Cependant, ils figurent explicitement au programme de Première TI et sont donc exigibles au Probatoire : tu dois connaître leur nom, leur rôle et leur syntaxe exacte.

**4.3 La légende du tableau : <caption>****DÉFINITION** Balise <caption>

La balise `<caption>` `</caption>` définit la **légende** (le titre) d'un tableau. Elle doit être placée juste après la balise ouvrante `<table>`, avant la première ligne `<tr>`. Par défaut, la légende s'affiche centrée au-dessus du tableau.

```
<table border="1">
  <caption>Moyennes du premier trimestre – Première TI</caption>
  <tr><th>Eleve</th><th>Moyenne</th></tr>
  <tr><td>Ngo Bell</td><td>15,5</td></tr>
</table>
```

#### 4.4 Fusion de cellules : `colspan` et `rowspan`

**Intuition.** Regarde un emploi du temps : le créneau « Informatique » occupe souvent **deux heures consécutives** (deux lignes), et la case « Pause » s'étend parfois sur **plusieurs colonnes**. Un tableau réel n'est donc pas toujours une grille régulière : certaines cellules « mangent » leurs voisines. HTML offre deux attributs pour cela.

##### DÉFINITION Fusion de cellules

- **colspan** (appliqué à `<td>` ou `<th>`) : fusion horizontale. `colspan="n"` signifie que la cellule s'étend sur  $n$  colonnes vers la droite.
- **rowspan** (appliqué à `<td>` ou `<th>`) : fusion verticale. `rowspan="n"` signifie que la cellule s'étend sur  $n$  lignes vers le bas.

Après une fusion, les cellules « absorbées » ne s'écrivent plus dans le code.

|                 |                 |   |   |
|-----------------|-----------------|---|---|
|                 | 1 (colspan="3") |   |   |
| 2 (rowspan="2") | 3               | 4 |   |
|                 | 5               | 6 |   |
|                 | 7               | 8 | 9 |

##### MÉTHODE Construire un tableau avec fusions

1. Dessiner le tableau sur papier, avec ses bordures et ses fusions.
2. Compter les colonnes de la ligne la plus large (celle sans fusion horizontale) : c'est le nombre de colonnes du tableau.
3. Repérer les fusions : pour chaque cellule élargie, noter son `colspan` ; pour chaque cellule étirée vers le bas, noter son `rowspan`.
4. Coder ligne par ligne, de haut en bas, et dans chaque ligne de gauche à droite, en n'écrivant que les cellules qui commencent dans cette ligne.
5. Vérifier : pour chaque ligne, la somme (nombre de cellules écrites + colonnes absorbées par les `colspan` de la ligne + colonnes occupées par les `rowspan` venus d'en haut) doit égaler le nombre total de colonnes.

Appliquons cette méthode à la figure ci-dessus (4 lignes, 3 colonnes) :

```

<table border="1" cellpadding="6">
  <tr>
    <th colspan="3">1 - Titre general</th>
  </tr>
  <tr>
    <td rowspan="2">2</td>
    <td>3</td>
    <td>4</td>
  </tr>
  <tr>
    <!-- la cellule 2 occupe deja la 1re colonne -->
    <td>5</td>
    <td>6</td>
  </tr>
  <tr>
    <td>7</td>
    <td>8</td>
    <td>9</td>
  </tr>
</table>

```

**ATTENTION** Erreur classique de comptage après un `colspan` ou un `rowspan`

Après un `colspan="3"`, la ligne contient moins de balises `<td>` : une seule cellule en remplace trois. De même, sous une cellule en `rowspan="2"`, la ligne suivante « saute » la colonne occupée. Si tu écris quand même toutes les cellules, le navigateur décale tout le tableau et crée une colonne fantôme à droite. Compte toujours en colonnes, jamais en nombre de balises.

**EXERCICE RÉSOLU 4** Bulletin de notes : tableau des moyennes

Énoncé : on veut afficher les moyennes de 3 élèves sur 4 matières (Maths, Physique, Informatique, Français), avec une ligne de titre fusionnée sur toute la largeur et une dernière colonne « Moyenne générale ». Écrire le code HTML.

**SOLUTION**

Solution détaillée : la ligne la plus large contient 6 colonnes (Nom + 4 matières + Moyenne). La ligne de titre utilise donc `colspan="6"`.

```

<table border="1" width="90%" align="center"
  cellpadding="6" cellspacing="0">
  <caption>Bulletin du 1er trimestre - Classe de Premiere TI</caption>
  <tr>
    <th colspan="6">College Bilingue de Yaounde - Releve de moyennes</th>
  </tr>
  <tr>
    <th>Eleve</th>
    <th>Maths</th>
    <th>Physique</th>

```

```

<th>Informatique</th>
<th>Francais</th>
<th>Moyenne</th>
</tr>
<tr>
<td>Etoundi A.</td>
<td>14</td><td>12</td><td>17</td><td>11</td>
<td>13,5</td>
</tr>
<tr>
<td>Kamga B.</td>
<td>11</td><td>13</td><td>15</td><td>14</td>
<td>13,25</td>
</tr>
<tr>
<td>Ngo Bell C.</td>
<td>16</td><td>15</td><td>18</td><td>13</td>
<td>15,5</td>
</tr>
</table>

```

Vérification : la ligne de titre compte 1 cellule × colspan="6" = 6 colonnes ; chaque ligne de données compte 6 cellules = 6 colonnes. Le tableau est cohérent.

#### EXPÉRIENCE TP guidé : réaliser l'emploi du temps de la classe

Objectif : créer une page `emploi.html` affichant l'emploi du temps de la Première TI avec fusions.

1. Ouvre le Bloc-notes (ou Notepad++) et crée le squelette HTML avec `<meta charset="utf-8">` dans `<head>`.
2. Sur papier, dessine le tableau : 6 colonnes (Heure, Lundi, ..., Vendredi) et 5 lignes de créneaux.
3. Le créneau « Informatique » du mardi dure 2 heures : code-le avec `rowspan="2"` et supprime la cellule correspondante de la ligne suivante.
4. La ligne « Pause de midi » s'étend sur toute la journée : code-la avec `<td colspan="6">`.
5. Ajoute `border="1"`, `cellpadding="8"`, `align="center"` et une balise `<caption>`.
6. Enregistre, ouvre le fichier dans le navigateur et vérifie que chaque ligne occupe bien 6 colonnes.

Observation attendue : les cellules fusionnées s'affichent élargies, sans décalage des lignes suivantes.

**EXERCICE 4 À toi de jouer**

Code un tableau « Résultats du championnat » de 4 colonnes (Équipe, Joués, Gagnés, Points) pour 3 équipes, avec une ligne de titre fusionnée (`colspan="4"`) et une cellule « Phase finale » en `rowspan="3"` dans une 5<sup>e</sup> colonne. Vérifie le comptage des colonnes ligne par ligne.

**À RETENIR**

Un tableau HTML se construit avec `<table>` (tableau), `<tr>` (ligne), `<td>` (cellule) et `<th>` (en-tête). Les attributs `border`, `width`, `height`, `align`, `cellpadding` (marge intérieure) et `cellspacing` (espace entre cellules) règlent son apparence. `colspan` fusionne horizontalement, `rowspan` verticalement ; après une fusion, les cellules absorbées ne s'écrivent pas. Méthode d'or : dessiner d'abord, compter les colonnes de la ligne la plus large, puis coder ligne par ligne en vérifiant le comptage.

## 5 LES FORMULAIRES

Dans la section précédente, nous avons appris à présenter des données sous forme de tableaux : l'information circulait dans un seul sens, du concepteur de la page vers le visiteur. Mais sur le Web réel, le visiteur n'est pas qu'un lecteur : il remplit des formulaires d'inscription à l'examen du Probatoire sur une plateforme en ligne, il saisit son numéro de téléphone pour recharger son compte mobile, il répond à un sondage. Comment une page web peut-elle recueillir des informations auprès de l'utilisateur ? C'est exactement le rôle des **formulaires** HTML, dernier grand mécanisme d'interaction que nous étudions cette année.

### 5.1 Définition et rôle d'un formulaire

Imagine le secrétariat de ton lycée au moment des inscriptions : on te remet une fiche papier comportant des cases à remplir (nom, date de naissance), des cases à cocher (sexe : M ou F), une liste de classes parmi lesquelles choisir, puis tu déposes la fiche remplie au secrétariat qui la traite. Un formulaire HTML fonctionne exactement pareil : la page joue le rôle de la fiche, les champs jouent le rôle des cases, et le bouton d'envoi joue le rôle du dépôt au secrétariat.

**DÉFINITION Formulaire**

Un **formulaire** est une zone d'une page web permettant de collecter des données saisies ou choisies par l'utilisateur (texte, mot de passe, choix unique ou multiple, etc.) en vue de les transmettre pour traitement. En HTML, un formulaire est délimité par la balise `<form> ... </form>`, qui contient les différents champs (zones de saisie, boutons, listes) et les bou-

tons d'envoi.

**DÉFINITION** La balise `<form>` et ses attributs

La balise `<form>` admet deux attributs essentiels :

- **action** : indique l'URL de la page vers laquelle les données saisies seront envoyées lorsque l'utilisateur valide le formulaire ;
- **method** : indique comment les données sont transmises. La valeur `get` place les données dans l'URL (visible, adaptée aux petites quantités de données non confidentielles) ; la valeur `post` les envoie dans le corps de la requête (invisible dans l'URL, adaptée aux données confidentielles comme un mot de passe).

Syntaxe générale : `<form action="page_cible" method="get">` ou `method="post"`.

**ATTENTION** Notion admise à ce niveau

En Première TI, on ne programme pas encore le traitement des données côté serveur : ce sera l'objet de la classe de Terminale (PHP). Les attributs `action` et `method` sont donc admis : tu dois savoir les écrire et expliquer leur rôle, sans savoir encore construire la page de traitement.

## 5.2 Les champs de saisie : la balise `<input>`

La balise `<input>` est une balise **orpheline** (sans balise fermante) dont le comportement dépend entièrement de son attribut **type**. C'est le couteau suisse du formulaire.

**PROPRIÉTÉ** Attributs communs de `<input>`

- **type** : nature du champ (`text`, `password`, `radio`, `checkbox`, `submit`, `reset`, ...);
- **name** : nom du champ, indispensable : c'est sous ce nom que la donnée sera transmise à la page cible ;
- **placeholder** : texte grisé affiché dans le champ vide pour guider l'utilisateur (il disparaît dès la saisie) ;
- **size** : largeur visible du champ, exprimée en nombre de caractères ;
- **value** : valeur initiale du champ, ou libellé d'un bouton.

Les deux champs de saisie les plus simples sont :

```
<!-- Champ texte ordinaire : ce qui est tapé reste visible -->
Nom : <input type="text" name="nom" placeholder="Ex : MBALLA" size="30">

<!-- Champ mot de passe : les caractères sont masqués par des points -->
Mot de passe : <input type="password" name="mdp" size="20">
```

La seule différence entre `text` et `password` est l'**affichage** : avec `password`, chaque caractère tapé est remplacé à l'écran par un point ou une étoile, pour qu'un voisin indiscret ne lise pas le mot de passe. Les données elles-mêmes sont transmises de la même façon.

### 5.3 Boutons radio et cases à cocher

Très souvent, on ne veut pas que l'utilisateur tape une réponse libre, mais qu'il **choisisse** parmi des propositions. Deux situations se présentent.

**Choix unique : les boutons radio.** Pour le sexe (M ou F), une seule réponse est possible. On utilise `<input type="radio">` et l'on **regroupe** les boutons en leur donnant le même attribut `name` : le navigateur comprend alors qu'ils s'excluent mutuellement — cocher l'un décoche automatiquement l'autre. L'attribut `value` précise la valeur transmise pour le bouton choisi.

Sexe :

```
<input type="radio" name="sexe" value="M"> Masculin
<input type="radio" name="sexe" value="F"> Feminin
```

#### ATTENTION Le piège classique des boutons radio

Si tu oublies de donner le même `name` à tous les boutons radio d'un groupe (par exemple `name="sexe1"` et `name="sexe2"`), le navigateur les considère comme indépendants : l'utilisateur pourra cocher plusieurs réponses, ce qui est absurde pour un choix unique. Vérifie toujours l'orthographe exacte du `name`, y compris les majuscules.

**Choix multiples : les cases à cocher.** Pour les options choisies par un élève (Informatique, Sport, Musique...), plusieurs réponses sont possibles simultanément. On utilise `<input type="checkbox">`; chaque case est indépendante et possède son propre `name`.

Options :

```
<input type="checkbox" name="info" value="oui"> Informatique
<input type="checkbox" name="sport" value="oui"> Sport
<input type="checkbox" name="musique" value="oui"> Musique
```

#### À RETENIR Radio ou case à cocher ?

Choix unique parmi plusieurs propositions ⇒ `radio` avec le même `name`. Choix multiples autorisés ⇒ `checkbox` avec des `name` différents.

### 5.4 La liste déroulante : `<select>` et `<option>`

Quand les propositions sont nombreuses (les 10 régions du Cameroun, les classes d'un lycée), aligner des boutons radio prendrait trop de place. On utilise alors une **liste déroulante**, construite avec la balise `<select>` (qui porte le `name`) contenant une balise `<option>` par choix possible. L'attribut **selected**, placé dans une `<option>`, désigne le choix présélectionné par défaut.

Classe :

```
<select name="classe">
  <option value="2nde">Seconde</option>
  <option value="1ereTI" selected>Première TI</option>
  <option value="1leTI">Terminale TI</option>
</select>
```

À l'affichage, seule l'option sélectionnée est visible ; un clic déroule la liste complète. Ici, « Première TI » apparaîtra choisie par défaut grâce à `selected`.

### 5.5 Étiquettes `<label>` et zone de texte multiligne

#### DÉFINITION La balise `<label>`

Une **étiquette** `<label>` associe un texte descriptif à un champ. On la lie au champ grâce à son attribut `for`, qui doit reprendre la valeur de l'attribut `id` du champ. Avantage pratique : cliquer sur le texte de l'étiquette active le champ (ou coche la case), ce qui améliore le confort de l'utilisateur.

```
<label for="nom">Nom du candidat :</label>
<input type="text" id="nom" name="nom" size="30">
```

Pour les réponses longues (adresse, motivation, commentaire), un champ `text` d'une seule ligne ne suffit pas : on utilise la zone de texte multiligne **`<textarea>`**, dont on fixe les dimensions avec les attributs `rows` (nombre de lignes) et `cols` (largeur en caractères). Attention : contrairement à `<input>`, `<textarea>` possède une balise fermante.

```
<label for="motiv">Vos motivations :</label><br>
<textarea id="motiv" name="motivations" rows="4" cols="40"></textarea>
```

### 5.6 Les boutons cliquables

Un formulaire sans bouton est une fiche qu'on ne peut pas déposer. Trois boutons sont à connaître :

#### PROPRIÉTÉ Boutons d'un formulaire

- `<input type="submit" value="Envoyer">` : valide le formulaire et transmet les données vers la page indiquée par l'attribut `action` du `<form>` ;
- `<input type="reset" value="Annuler">` : efface toutes les saisies et ramène le formu-

laire à son état initial ;

- `<button type="submit">Envoyer</button>` : variante à balise fermante, dont le libellé est le contenu de la balise (elle peut contenir du texte enrichi ou une image).

#### MÉTHODE Construire un formulaire propre et lisible

1. Ouvrir la balise `<form>` avec ses attributs `action` et `method`.
2. Pour chaque information à collecter, écrire un label (ou un texte) + un champ par ligne, en séparant les lignes par `<br>` ou en les plaçant dans des paragraphes.
3. Choisir le bon type de champ : `text/password` pour une saisie libre, `radio` pour un choix unique (même `name` pour tout le groupe!), `checkbox` pour des choix multiples, `select` pour une longue liste.
4. Donner à chaque champ un `name` explicite et unique (sauf groupe radio).
5. Terminer par les boutons `submit` et `reset`, puis fermer `</form>`.
6. Tester dans le navigateur : remplir, vérifier que les radios s'excluent, que le bouton Annuler efface tout.

#### EXEMPLE Formulaire complet « Concours d'entrée en 1ère TI »

Voici le code intégral d'un formulaire d'inscription, conforme à la maquette de la figure ci-dessous :

```
<form action="inscription.html" method="post">
  <label for="nom">Nom et prenom :</label>
  <input type="text" id="nom" name="nom"
    placeholder="Ex : NGOUEFO Amina" size="30"><br>

  Sexe :
  <input type="radio" name="sexe" value="M" id="m">
  <label for="m">Masculin</label>
  <input type="radio" name="sexe" value="F" id="f">
  <label for="f">Feminin</label><br>

  <label for="classe">Classe demandee :</label>
  <select id="classe" name="classe">
    <option value="1ereTI" selected>Premiere TI</option>
    <option value="1ereD">Premiere D</option>
    <option value="1ereC">Premiere C</option>
  </select><br>

  Options souhaitees :
  <input type="checkbox" name="sport" value="oui"> Sport
  <input type="checkbox" name="club" value="oui"> Club informatique<br>
```

```
<input type="submit" value="Envoyer">
<input type="reset" value="Annuler">
</form>
```

**Formulaire d'inscription au lycée**

Nom et prenom :  <input type="text">

Sexe : ☐ Masculin ☒ Féminin <input type="radio"> même name

Classe :  <select> + <option>

Options ☒ Sport ☐ Club informatique <input type="checkbox">

type="submit" type="reset"

<form action="inscription.html" method="post">

Maquette annotée du formulaire d'inscription : chaque élément visuel correspond à une balise HTML précise.

## 5.7 Que deviennent les données ? Trajet et limites

Lorsque l'utilisateur clique sur le bouton `submit`, le navigateur rassemble les couples `name=valeur` de tous les champs remplis et les transmet à l'URL indiquée par `action`, selon la méthode précisée par `method`. La figure suivante résume ce trajet.



Trajet des données : le bouton `submit` déclenche l'envoi des données vers la page indiquée par l'attribut `action`.

### ATTENTION Limite fondamentale du HTML seul

En HTML seul, les données d'un formulaire ne sont ni vérifiées (on peut saisir un nom vide

ou une date impossible) ni enregistrées (aucune base de données n'est interrogée). Le HTML ne fait que présenter le formulaire et transmettre les données. Le contrôle des saisies se fera en JavaScript (côté client) et leur traitement définitif (enregistrement, réponse personnalisée) exigera un langage serveur comme PHP, étudié en Terminale TI. Ne dis donc jamais qu'un formulaire HTML « enregistre » des informations : il les envoie.

#### EXERCICE RÉSOLU 5 Corriger un formulaire défectueux

Énoncé. Un élève de Première TI écrit le fragment suivant pour demander le sexe du visiteur, mais on constate qu'on peut cocher les deux réponses à la fois. De plus, rien ne se passe au clic sur « Envoyer ». Identifier les deux erreurs et corriger le code.

```
<form>
  Sexe :
  <input type="radio" name="sexe1" value="M"> Masculin
  <input type="radio" name="sexe2" value="F"> Feminin
  <input type="submit" value="Envoyer">
</form>
```

#### SOLUTION

Solution détaillée. Erreur 1 : les deux boutons radio portent des `name` différents (`sexe1` et `sexe2`). Le navigateur les traite donc comme deux groupes indépendants et autorise deux coches simultanées. Il faut le même `name` pour tout le groupe. Erreur 2 : la balise `<form>` n'a pas d'attribut `action` : le navigateur ne sait pas vers quelle page envoyer les données. On ajoute `action` (et `method` par bonne pratique). Code corrigé :

```
<form action="traitement.html" method="post">
  Sexe :
  <input type="radio" name="sexe" value="M"> Masculin
  <input type="radio" name="sexe" value="F"> Feminin
  <input type="submit" value="Envoyer">
</form>
```

Désormais, les deux boutons s'excluent mutuellement et le clic sur « Envoyer » transmet le couple `sexe=M` (ou `sexe=F`) vers `traitement.html`.

#### EXPÉRIENCE TP guidé : créer le formulaire d'inscription du lycée

Objectif : réaliser et tester le formulaire complet de cette leçon. Matériel et logiciels : un PC, le Bloc-notes (ou Notepad++), un navigateur (Chrome ou Firefox). Manipulation :

1. Crée un dossier `mon_site` puis un fichier `inscription.html` ; écris la structure de base d'une page HTML avec `<meta charset="utf-8">` dans le `<head>`.

2. Dans le `<body>`, recopie le formulaire complet de l'exemplebox « Concours d'entrée en 1ère TI ».
3. Enregistre, puis ouvre le fichier dans le navigateur (double-clic).
4. Remplis le formulaire : vérifie que les deux boutons radio s'excluent, que plusieurs cases peuvent être cochées, que la liste déroulante propose les trois classes avec « Première TI » présélectionnée.
5. Clique sur « Annuler » : toutes les saisies doivent s'effacer. Clique sur « Envoyer » : le navigateur tente d'ouvrir la page cible de `action`.

Observations : les radios se comportent comme un choix unique, les cases à cocher comme des choix multiples ; le bouton `reset` réinitialise le formulaire. Interprétation : le comportement des champs dépend uniquement des attributs `type` et `name` ; l'envoi est déclenché par `submit` vers l'URL de `action`, mais aucun traitement réel n'a lieu tant qu'aucun langage serveur n'est utilisé.

#### EXERCICE 5 À toi de jouer

Le cybercafé « Espace Numérique » de Bafoussam veut un formulaire de commande en ligne. Écris le code HTML d'un formulaire (méthode `post`, action `commande.html`) comportant : le nom du client (champ texte avec placeholder), le type de service demandé (boutons radio : « Impression », « Photocopie », « Saisie »), les options payantes (cases à cocher : « Reliure », « Plastification »), la commune de livraison (liste déroulante : Bafoussam I, II, III — Bafoussam I présélectionnée), un commentaire libre (`textarea` de 3 lignes), et les boutons « Commander » et « Effacer ». Chaque champ devra avoir un `name` correct et les radios devront former un seul groupe.

#### CORRIGÉ Exercice : formulaire du cybercafé

```
<form action="commande.html" method="post">
  <label for="client">Nom du client :</label>
  <input type="text" id="client" name="client"
    placeholder="Ex : TCHOUP O Eric" size="30"><br>

  Service :
  <input type="radio" name="service" value="impression"> Impression
  <input type="radio" name="service" value="photocopie"> Photocopie
  <input type="radio" name="service" value="saisie"> Saisie<br>

  Options :
  <input type="checkbox" name="reliure" value="oui"> Reliure
  <input type="checkbox" name="plastification" value="oui"> Plastification<br>
```

```
<label for="commune">Commune de livraison :</label>
<select id="commune" name="commune">
  <option value="baf1" selected>Bafoussam I</option>
  <option value="baf2">Bafoussam II</option>
  <option value="baf3">Bafoussam III</option>
</select><br>

<label for="com">Commentaire :</label><br>
<textarea id="com" name="commentaire" rows="3" cols="40"></textarea><br>

<input type="submit" value="Commander">
<input type="reset" value="Effacer">
</form>
```

Points de vigilance : un seul `name="service"` pour les trois radios ; des `name` distincts pour les deux cases à cocher ; `selected` sur Bafoussam I ; balise fermante pour `<textarea>`.

#### À RETENIR L'essentiel sur les formulaires

- Un formulaire (`<form action="..." method="get|post">`) collecte des données saisies par l'utilisateur et les transmet à une page cible.
- `<input>` change de rôle selon `type` : `text`, `password` (saisie masquée), `radio` (choix unique, même `name`), `checkbox` (choix multiples), `submit` (envoyer), `reset` (effacer).
- Liste déroulante : `<select>` + `<option>`, avec `selected` pour le choix par défaut ; texte long : `<textarea rows cols>`.
- `<label for="id">` relie un texte à son champ ; `name` nomme la donnée transmise.
- HTML seul ne vérifie ni n'enregistre rien : le traitement relève d'un langage serveur (Terminale).

## 6 ORGANISATION D'UN SITE, BALISES DE SECTION ET LIMITES DU HTML

Dans la section précédente, nous avons appris à insérer des **formulaires** dans une page web : champs de saisie, boutons radio, cases à cocher, listes déroulantes. Nous savons donc désormais construire des pages riches. Mais un vrai site web, comme celui de ton lycée ou d'une boutique en ligne, ne tient pas dans un seul fichier : il contient des dizaines de pages, des centaines d'images, des fichiers de style. Comment ranger tout cela proprement ? Comment regrouper les zones d'une page (en-tête, menu, contenu, pied de page) ? Et surtout : jusqu'où le HTML peut-il aller seul ? C'est l'objet de cette dernière section.

## 6.1 Organiser les dossiers d'un site web

### Pourquoi organiser ?

Imagine un élève qui range tous ses cahiers, ses stylos, ses brouillons et ses manuels en vrac dans un seul sac : le jour de l'interrogation, il ne retrouve rien. C'est exactement ce qui arrive à un développeur qui place tous ses fichiers (pages HTML, images, feuilles de style) dans un seul dossier. Dès que le site dépasse cinq ou six fichiers, il devient introuvable et impossible à maintenir. La solution est une **arborescence** : une organisation hiérarchique en dossiers et sous-dossiers.

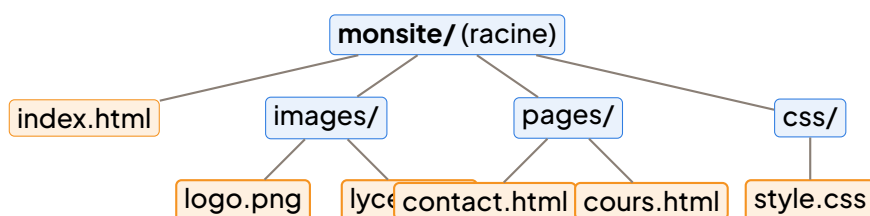
#### DÉFINITION Dossier racine et index.html

Le **dossier racine** est le dossier principal qui contient tous les fichiers et sous-dossiers du site. Le fichier `index.html`, placé dans la racine, est la **page d'accueil** : c'est le fichier que le serveur web affiche automatiquement lorsqu'un visiteur tape l'adresse du site sans préciser de page.

#### MÉTHODE Organiser les dossiers d'un site web avant de coder

Avant d'écrire la moindre ligne de HTML, suis ces étapes :

1. Crée le dossier racine du site, par exemple `monsite/`.
2. À l'intérieur, crée les sous-dossiers : `images/` pour les photos et logos, `css/` pour les feuilles de style, `pages/` pour les pages secondaires.
3. Place le fichier `index.html` (page d'accueil) directement dans la racine.
4. Place chaque fichier dans le dossier qui correspond à son rôle : une photo dans `images/`, une page secondaire dans `pages/`.
5. Nomme les fichiers en minuscules, sans espaces ni accents : `photo_classe.jpg` et non `Photo de la classe.JPG`.



Arborescence type d'un petit site web : la racine contient la page d'accueil et trois sous-dossiers spécialisés.

## 6.2 Les chemins relatifs dans une arborescence

Une fois les fichiers rangés dans des dossiers différents, il faut pouvoir les désigner les uns depuis les autres. On utilise pour cela des **chemins relatifs**, c'est-à-dire des adresses exprimées par rapport au fichier courant.

### DÉFINITION Chemin relatif

Un **chemin relatif** indique la position d'un fichier par rapport au dossier contenant la page en cours. On utilise :

- `nomfichier` pour un fichier du dossier courant ;
- `dossier/nomfichier` pour descendre dans un sous-dossier ;
- `../nomfichier` pour remonter vers le dossier parent (`../` signifie « dossier au-dessus »).

### EXEMPLE Chemins relatifs en pratique

Avec l'arborescence de la figure précédente :

- Depuis `index.html`, afficher le logo : `images/logo.png` (on descend dans `images/`).
- Depuis `pages/contact.html`, afficher le logo : `../images/logo.png` (on remonte à la racine, puis on descend dans `images/`).
- Depuis `pages/contact.html`, revenir à l'accueil : `../index.html`.

```
<!-- Dans pages/contact.html : retour vers l'accueil -->
<a href="../index.html" title="Retour a l'accueil">Accueil</a>

<!-- Dans pages/contact.html : image situee dans images/ -->

```

### ATTENTION Le piège du dossier parent oublié

L'erreur la plus fréquente : une image s'affiche bien depuis `index.html` mais devient « cassée » (icône d'image brisée) dans une page du dossier `pages/`. La cause est presque toujours l'oubli du `../`. Vérifie toujours : « où suis-je ? où est le fichier visé ? ». Autre piège : les espaces et accents dans les noms de fichiers, qui cassent les liens sur certains serveurs.

## 6.3 Les balises de section : DIV, SECTION et SPAN

Regarde une page de journal : il y a un titre principal, des colonnes, un encadré publicité, un pied de page. Une page web est organisée de la même façon en **zones** : en-tête, menu, contenu

principal, pied de page. Pour délimiter ces zones, le HTML propose des **balises de section**.

**DÉFINITION** Balises de section

- `<DIV>` : **conteneur de type bloc** générique. Il regroupe plusieurs éléments (titres, paragraphes, images) dans une zone rectangulaire, sans signification particulière. C'est la balise de structuration la plus utilisée.
- `<SECTION>` : conteneur de type bloc qui regroupe des éléments formant un ensemble thématique cohérent (un chapitre, une rubrique « Actualités », une fiche produit). Contrairement à `<DIV>`, elle a une signification sémantique.
- `<SPAN>` : **conteneur en ligne**. Il encadre un petit morceau de contenu à l'intérieur d'un texte (un mot, une phrase) sans provoquer de retour à la ligne, par exemple pour colorer un seul mot.

**Élément bloc ou élément en ligne ?**

C'est une distinction fondamentale en HTML. Un **élément bloc** (`<div>`, `<p>`, `<h1>`, `<section>`, `<table>`) occupe toute la largeur disponible et provoque un retour à la ligne avant et après lui : il forme un « bloc » empilé verticalement. Un **élément en ligne** (`<span>`, `<a>`, `<img>`, `<b>`) s'insère dans le flux du texte, comme un mot dans une phrase, sans retour à la ligne.

**EXEMPLE** Structurer une page avec DIV, SECTION et SPAN

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Site du Lycee de Bafoussam</title>
</head>
<body>
  <!-- En-tete du site : bloc generique -->
  <div id="entete">
    <h1>Lycee de Bafoussam</h1>
    <p>Bienvenue sur le site officiel de notre etablissement. </p>
  </div>

  <!-- Rubrique thematique : balise SECTION -->
  <section id="actualites">
    <h2>Actualites</h2>
    <p>La rentree scolaire aura lieu le
      <span class="important">8 septembre</span>. </p>
  </section>

  <!-- Pied de page -->
  <div id="pied">
    <p>Contact : lycee.bafoussam@education.cm</p>
  </div>
</body>
```

```
</html>
```

Ici, `<span class="important">` encadre uniquement la date, à l'intérieur de la phrase : c'est un conteneur en ligne. Les `<div>` et `<section>` découpent la page en grandes zones.

#### LESAVAIS-TU ? HTML5 et les balises sémantiques

HTML5 a introduit des balises de section plus précises : `<header>` (en-tête), `<footer>` (pied de page), `<nav>` (navigation), `<main>` (contenu principal), `<article>` (contenu autonome), `<aside>` (contenu annexe). Elles remplacent avantageusement `<div>` quand leur sens correspond, car elles améliorent le référencement (SEO) et l'accessibilité. Le programme de Première TI exige la maîtrise de `<DIV>`, `<SECTION>` et `<SPAN>`; ces nouvelles balises seront approfondies en Terminale.

## 6.4 Repérer les sections : les attributs `id` et `class`

Pour identifier les zones d'une page (afin de les styliser en CSS ou de les manipuler en JavaScript), on utilise deux attributs universels, valables sur presque toutes les balises.

#### DÉFINITION Attributs `id` et `class`

- L'attribut **id** donne un identifiant unique à un élément : deux éléments d'une même page ne peuvent jamais porter le même `id`. Exemple : `<div id="entete">`.
- L'attribut **class** attribue une étiquette réutilisable : plusieurs éléments peuvent partager la même classe, et un élément peut avoir plusieurs classes (séparées par des espaces). Exemple : `<p class="important">` ou `<div class="carte produit">`.

#### ATTENTION `id` unique, `class` réutilisable

Écrire deux fois `id="menu"` dans la même page est une erreur : le navigateur ne saura plus quel élément viser, et le JavaScript ou le CSS associé se comportera de façon imprévisible. Règle simple : un `id` est comme un numéro de CNI (unique), une `class` est comme un uniforme (porté par plusieurs élèves).

| Critère       | id                                      | class                                               |
|---------------|-----------------------------------------|-----------------------------------------------------|
| Unicité       | Unique dans toute la page               | Réutilisable à volonté                              |
| Usage typique | Repérer une zone précise (entête, pied) | Regrouper des éléments semblables (important, note) |
| Exemple       | <code>&lt;div id="entete"&gt;</code>    | <code>&lt;span class="important"&gt;</code>         |

## 6.5 Les limites du HTML

Nous avons vu que le HTML permet de structurer le contenu : titres, paragraphes, listes, liens, médias, tableaux, formulaires, sections. Mais essaie de réaliser avec le seul HTML un site aux couleurs de ton lycée, avec un menu animé et un calcul automatique de moyenne : tu n'y arriveras pas. Ce n'est pas un oubli des concepteurs, c'est un choix.

### PROPRIÉTÉ Limites du HTML (admise)

Le HTML est un langage de description de contenu, pas un langage de programmation. À lui seul, il ne permet :

- ni mise en forme évoluée (couleurs, polices, mise en page précise, animations visuelles) : c'est le rôle du **CSS** ;
- ni interactivité (réagir à un clic, vérifier une saisie, animer un menu, calculer dans la page) : c'est le rôle du **JavaScript** ;
- ni traitement ni stockage de données (enregistrer un client, gérer des comptes, interroger une base de données) : c'est le rôle des langages serveur (comme PHP) associés à une base de données.

JavaScript — l'interactivité (le « comportement »)

CSS — la présentation et les couleurs (la « forme »)

HTML — la structure et le contenu (le « fond »)

→ Une page web moderne : 3 couches

Les trois couches d'une page web : HTML structure le contenu, CSS le met en forme, JavaScript le rend interactif.

## 6.6 Site statique et site dynamique : bilan

### DÉFINITION Site statique et site dynamique

Un **site statique** est constitué de pages HTML écrites une fois pour toutes : tous les visiteurs voient exactement le même contenu, qui ne change que si le webmestre modifie les fichiers. Un **site dynamique** génère ses pages à la demande, à partir d'une base de données : le contenu s'adapte au visiteur (son compte, son panier, l'heure, ses recherches).

| Critère      | Site statique                         | Site dynamique                                      |
|--------------|---------------------------------------|-----------------------------------------------------|
| Technologies | HTML + CSS (+ JavaScript côté client) | HTML + CSS + JS + langage serveur + base de données |
| Contenu      | Identique pour tous, figé             | Personnalisé, généré à la demande                   |
| Mise à jour  | Modification manuelle des fichiers    | Via une interface d'administration                  |
| Exemples     | Site vitrine d'un lycée, CV en ligne  | Jumia, Facebook, site de notes en ligne             |

#### LES AVAIS-TU ? Les grands sites sont dynamiques

Les sites que tu utilises chaque jour — Jumia, Facebook, WhatsApp Web, les plateformes de paiement mobile — sont des sites dynamiques : quand tu ouvres Facebook, la page affiche ton fil d'actualité, différent de celui de ton voisin. Un site statique, lui, affiche le même contenu à tous les visiteurs, comme un panneau d'affichage. C'est parfait pour présenter un lycée, une entreprise ou un événement, et c'est exactement ce que tu sais maintenant construire !

#### EXERCICE RÉSOLU 6 Organiser le site d'une bibliothèque scolaire

Énoncé. La bibliothèque du lycée veut un site statique de trois pages : un accueil, une page « Catalogue » et une page « Contact ». Les images (logo, photo de la salle) et la feuille de style doivent être rangées séparément. Propose une arborescence, puis écris le code du lien et de l'image à placer dans la page `catalogue.html` pour revenir à l'accueil et afficher le logo.

#### SOLUTION

Solution détaillée. 1. Arborescence. On crée la racine `bibliotheque/` contenant `index.html` et trois sous-dossiers : `images/` (`logo.png`, `salle.jpg`), `pages/` (`catalogue.html`, `contact.html`), `css/` (`style.css`).

2. Chemins relatifs. Le fichier `catalogue.html` se trouve dans `pages/`. Pour atteindre la racine, on remonte avec `../` ; pour atteindre le logo, on remonte puis on descend dans `images/` :

```
<!-- Dans pages/catalogue.html -->
<a href="../index.html" title="Retour à l'accueil">Accueil</a>

```

3. Vérification. Sans le `../`, le navigateur chercherait `pages/index.html` et `pages/images/logo.png`, qui n'existent pas : lien mort et image cassée.

**EXPÉRIENCE** TP : construire l'arborescence d'un mini-site

Objectif : créer l'arborescence complète d'un site statique et vérifier les chemins relatifs. Matériel : un PC avec un éditeur de texte (Bloc-notes, Notepad++) et un navigateur. Manipulation :

1. Crée le dossier `monsite/` puis les sous-dossiers `images/`, `pages/` et `css/`.
2. Place une image quelconque dans `images/` sous le nom `photo.jpg`.
3. Crée `index.html` à la racine avec un lien vers `pages/page2.html` et l'image `images/photo.jpg`.
4. Crée `pages/page2.html` contenant un lien de retour `../index.html` et l'image `../images/photo.jpg`.
5. Ouvre `index.html` dans le navigateur, clique le lien, vérifie que l'image s'affiche sur les deux pages.

Observations : si l'image n'apparaît pas sur la page 2, c'est que le `../` manque. Corrige et recharge (touche F5). Interprétation : un chemin relatif se lit toujours depuis le fichier qui le contient, jamais depuis la racine.

**EXERCICE 6** À toi de jouer

1. Un site contient `index.html` à la racine et `pages/infos.html`. Quel chemin relatif faut-il écrire dans `infos.html` pour afficher `css/style.css` ?
2. Pourquoi est-il interdit d'avoir deux `<div id="menu">` dans la même page ?
3. Cite les trois limites du HTML et le langage qui compense chacune d'elles.
4. Le site des résultats du Probatoire, où chaque candidat consulte sa note, est-il statique ou dynamique ? Justifie.

**CORRIGÉ** Exercice

1. `../css/style.css` : on remonte de `pages/` vers la racine avec `../`, puis on descend dans `css/`.
2. Un `id` doit être unique dans une page : avec deux identifiants identiques, le navigateur ne sait plus quel élément cibler, et le CSS ou le JavaScript associé devient imprévisible. Pour repérer plusieurs éléments semblables, on utilise `class`.
3. Pas de mise en forme évoluée → CSS ; pas d'interactivité → JavaScript ; pas de traitement ni stockage de données → langages serveur (PHP) et bases de données.

4. C'est un site dynamique : le contenu affiché dépend du candidat connecté et provient d'une base de données. Un site statique afficherait la même page à tous.

**À RETENIR****L'essentiel de la section**

- Un site s'organise en arborescence : racine contenant `index.html` et des sous-dossiers (`images/`, `css/`, `pages/`).
- Les chemins relatifs désignent un fichier depuis la page courante : `dossier/fichier` pour descendre, `../` pour remonter au dossier parent.
- `<DIV>` et `<SECTION>` sont des conteneurs bloc (zones de page) ; `<SPAN>` est un conteneur en ligne (morceau de texte).
- `id` est unique dans la page ; `class` est réutilisable.
- Le HTML seul ne fait que structurer : la forme revient au CSS, l'interactivité au JavaScript, le traitement des données aux langages serveur et aux bases de données. Un site statique affiche le même contenu à tous ; un site dynamique le génère à la demande.

Tu sais désormais construire un site statique complet et bien organisé. La prochaine étape de ton parcours ? Donner du style à tes pages avec le CSS, puis les faire réagir avec JavaScript : c'est la séparation fond / forme / comportement, la règle d'or du web moderne.

## 7 ACTIVITÉ D'INTÉGRATION

## 8 LEÇON N05 : PROGRAMMATION WEB STATIQUE EN HTML

### 8.1 Site web, HTML et structure d'une page

#### DÉFINITION Site web et page web

Un **site web** est un ensemble de fichiers (pages HTML, images, styles, scripts) liés entre eux par des hyperliens, hébergés sur un serveur web et accessibles via une **URL** (Uniform Resource Locator). Une **page web** est un document unique, rédigé en **HTML** (HyperText Markup Language), que le navigateur interprète pour l'afficher à l'écran.

#### PROPRIÉTÉ Structure minimale d'un document HTML5

Tout document HTML5 valide respecte la structure suivante :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Titre de la page</title>
</head>
<body>
  <!-- Contenu visible de la page -->
</body>
</html>
```

- `<!DOCTYPE html>` : déclare la version HTML5 (obligatoire en première ligne).
- `<html lang="fr">` : racine du document, langue principale française.
- `<head>` : métadonnées (titre, encodage, styles, scripts) non affichées directement.
- `<meta charset="utf-8">` : indispensable pour afficher correctement les caractères accentués (é, è, à, ç, ô...) et spéciaux. Sans elle, le navigateur peut utiliser un encodage par défaut (ISO-8859-1) et afficher des caractères illisibles (≡).
- `<title>` : titre affiché dans l'onglet du navigateur et les favoris.
- `<body>` : contient tout le contenu visible (textes, images, tableaux, formulaires...).

**MÉTHODE** Créer une page HTML valide

1. Ouvrir un éditeur de texte simple (Notepad++, VS Code, Gedit...).
2. Écrire la structure minimale ci-dessus.
3. Enregistrer le fichier avec l'extension `.html` (ex. `index.html`), sans accents, sans espaces, en minuscules (ex. `page_accueil.html`).
4. Ouvrir le fichier dans un navigateur (double-clic ou Fichier > Ouvrir).

## 8.2 Listes, hyperliens et URL

**DÉFINITION** URL (Uniform Resource Locator)

Une **URL** est l'adresse unique d'une ressource sur le web. Elle se compose : `protocole://domaine/chemin/fichier?paramètres#ancree`. En travail local, on utilise des **chemins relatifs** :

- `fichier.html` : même dossier.
- `dossier/fichier.html` : descendre dans un sous-dossier.
- `../fichier.html` : remonter d'un niveau (dossier parent).

**PROPRIÉTÉ** Hyperlien – Balise `<a>`

La balise `<a>` (ancree) crée un lien hypertexte.

```
<a href="URL" title="Infobulle au survol" target="_blank">Texte ou image cliquable</a>
```

- `href` : destination (URL absolue `https://...` ou relative `pages/contact.html`).
- `title` : texte de l'**infobulle** affichée au survol de la souris (accessibilité, UX).
- `target="_blank"` : ouvre le lien dans un nouvel onglet (optionnel).

**PROPRIÉTÉ** Listes – `<ul>`, `<ol>`, `<li>`

- `<ul>` (unordered list) : liste à puces (ordre non important).
- `<ol>` (ordered list) : liste numérotée (ordre important).
- `<li>` (list item) : chaque élément.

- **Imbrication** : une liste complète (<ul> ou <ol>) se place à l'intérieur d'un <li> de la liste parente.

```
<ul>
  <li>Série Scientifique
    <ul>
      <li>C</li>
      <li>D</li>
    </ul>
  </li>
  <li>Série Technique
    <ul><li>TI</li></ul>
  </li>
</ul>
```

### 8.3 Insertion de médias : images, sons, vidéos

#### PROPRIÉTÉ Image – Balise <img> (balise orpheline / auto-fermante)

```

```

- **src** (source) : chemin relatif ou absolu vers le fichier image (PNG, JPG, GIF, SVG, WebP).
- **alt** (alternative) : obligatoire pour l'accessibilité (lecteurs d'écran) et le référencement ; affiché si l'image ne charge pas.
- **width, height** : dimensions en pixels (réserve l'espace, évite le « saut » de mise en page).

#### PROPRIÉTÉ Image cliquable

On imbrique la balise <img> à l'intérieur de la balise <a> :

```
<a href="index.html" title="Retour à l'accueil">
  
</a>
```

#### PROPRIÉTÉ Son et Vidéo – Balises <audio> et <video> (HTML5)

Ces balises permettent l'insertion native sans plugin externe (Flash obsolète).

```
<!-- Audio -->
<audio controls>
  <source src="media/son.mp3" type="audio/mpeg">
  <source src="media/son.ogg" type="audio/ogg">
  Votre navigateur ne supporte pas l'audio HTML5.
</audio>

<!-- Vidéo -->
<video controls width="640" height="360" poster="images/vignette.jpg">
  <source src="media/video.mp4" type="video/mp4">
  <source src="media/video.webm" type="video/webm">
  Votre navigateur ne supporte pas la vidéo HTML5.
</video>
```

- **controls** : affiche les boutons lecture/pause, volume, barre de progression.
- **<source>** : permet de proposer plusieurs formats (compatibilité navigateurs).
- **type** : type MIME (ex. video/mp4, audio/mpeg).
- **poster (vidéo)** : image d'attente avant lecture.

## 8.4 Les tableaux en HTML

### ATTENTION Attributs de présentation vs CSS

Le programme officiel impose la connaissance des attributs `border`, `width`, `height`, `align`, `cellpadding`, `cellspacing` sur `<table>`. Notez que ces attributs sont obsolètes (dépréciés) en HTML5 : le style (bordures, espacement, alignement, largeur) se gère normalement en CSS. Pour le Probatoire, vous devez savoir les utiliser en HTML pur.

### PROPRIÉTÉ Structure de base d'un tableau

```
<table border="1" width="60%" align="center" cellpadding="5" cellspacing="0" id="tab-resultats">
  <caption>Titre du tableau (optionnel)</caption>
  <tr>
    <!-- Table Row : ligne -->
    <th>En-tête 1</th> <!-- Table Header : cellule d'en-tête (gras, centré) -->
    <th>En-tête 2</th>
  </tr>
  <tr>
    <td>Donnée 1</td> <!-- Table Data : cellule standard -->
    <td>Donnée 2</td>
  </tr>
</table>
```

- **border="1"** : épaisseur de la bordure en pixels.

- `width="60%"` : largeur relative à la fenêtre ou au conteneur parent.
- `align="center"` : centre le tableau horizontalement (`left`, `right` possibles).
- `cellpadding="5"` : espace interne entre le bord de la cellule et son contenu (en pixels).
- `cellspacing="0"` : espace externe entre les cellules (bordures collées si 0).
- `id` : identifiant unique pour cibler le tableau en CSS/JS.

#### PROPRIÉTÉ Fusion de cellules – `colspan` et `rowspan`

- `colspan="n"` : fusionne `n` cellules horizontalement (sur une même ligne).
- `rowspan="n"` : fusionne `n` cellules verticalement (sur une même colonne).

Règle d'or : une cellule fusionnée disparaît des lignes/colonnes suivantes. Il faut retirer les balises `<td>` ou `<th>` correspondantes dans le code des lignes concernées.

#### EXEMPLE Tableau avec fusions (extrait de l'activité)

Ligne de titre sur 3 colonnes (`colspan="3"`), cellule « 2024 » sur 3 lignes (`rowspan="3"`).

```
<table border="1" width="60%" align="center" cellpadding="5" cellspacing="0">
  <tr>
    <th colspan="3">Résultats du Probatoire 2024</th>
  </tr>
  <tr>
    <th>Année</th><th>Série</th><th>Taux de réussite</th>
  </tr>
  <tr>
    <td rowspan="3">2024</td><td>C</td><td>78 %</td>
  </tr>
  <tr>
    <td>D</td><td>71 %</td>
  </tr>
  <tr>
    <td>TI</td><td>85 %</td>
  </tr>
</table>
```

## 8.5 Les formulaires

#### DÉFINITION Formulaire HTML

Un **formulaire** (`<form>`) est une zone de la page permettant à l'utilisateur de saisir des données pour les envoyer à un serveur (traitement côté serveur : PHP, Python, Node.js... vu en Terminale). En HTML statique, le formulaire s'affiche et se remplit, mais l'envoi (`submit`) ne produit aucun traitement visible sans script serveur.

**PROPRIÉTÉ** Balise `<form>` et attributs principaux

```
<form action="traitement.php" method="post">
  <!-- Contrôles de saisie -->
</form>
```

- `action` : URL du script serveur qui traitera les données (ex. `inscription.php`, `mailto:email@domaine.com` pour test).
- `method` : `get` (données dans l'URL, visible, limité) ou `post` (données dans le corps HTTP, invisible, sans limite de taille, recommandé pour les mots de passe et formulaires longs).

**PROPRIÉTÉ** Contrôles de saisie courants – `<input>`, `<select>`, `<textarea>`

| Type / Balise                                | Code exemple                                                                                                                                                     | Rôle / Attributs clés                                                                                                                   |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;input type="text"&gt;</code>       | <code>&lt;input type="text" name="nom" id="nom" size="30" placeholder="Votre nom"&gt;</code>                                                                     | Champ texte une ligne. <b>name</b> (clé envoyée au serveur), <b>id</b> (lien <code>&lt;label&gt;</code> ), <b>placeholder</b> (indice). |
| <code>&lt;input type="date"&gt;</code>       | <code>&lt;input type="date" name="naissance"&gt;</code>                                                                                                          | Sélecteur de date natif (calendrier). Format envoyé : AAAA-MM-JJ.                                                                       |
| <code>&lt;input type="radio"&gt;</code>      | <code>&lt;input type="radio" name="sexe" value="M" id="m"&gt; &lt;label for="m"&gt;Masculin&lt;/label&gt;</code>                                                 | Bouton radio : choix unique dans un groupe. Même <b>name</b> pour tous les boutons du groupe. <b>value</b> = valeur envoyée.            |
| <code>&lt;input type="checkbox"&gt;</code>   | <code>&lt;input type="checkbox" name="club" value="info" id="club"&gt; &lt;label for="club"&gt;Club info&lt;/label&gt;</code>                                    | Case à cocher : choix multiple (0, 1 ou plusieurs). <b>name</b> différent pour chaque case.                                             |
| <code>&lt;select&gt; + &lt;option&gt;</code> | <code>&lt;select name="serie" id="serie"&gt; &lt;option value="C"&gt;C&lt;/option&gt; &lt;option value="TI" selected&gt;TI&lt;/option&gt; &lt;/select&gt;</code> | Liste déroulante. <b>selected</b> = choix par défaut. multiple autorise plusieurs choix (avec Ctrl).                                    |
| <code>&lt;textarea&gt;</code>                | <code>&lt;textarea name="msg" rows="5" cols="40"&gt;&lt;/textarea&gt;</code>                                                                                     | Zone de texte multi-lignes.                                                                                                             |
| <code>&lt;input type="submit"&gt;</code>     | <code>&lt;input type="submit" value="Envoyer"&gt;</code>                                                                                                         | Bouton d'envoi du formulaire vers action.                                                                                               |
| <code>&lt;input type="reset"&gt;</code>      | <code>&lt;input type="reset" value="Effacer"&gt;</code>                                                                                                          | Bouton de remise à zéro des champs.                                                                                                     |

**MÉTHODE****Bonnes pratiques d'accessibilité pour les formulaires**

1. Associer chaque champ à un `<label>` via `for="id_du_champ"` (clic sur le label = focus sur le champ).
2. Regrouper les champs logiquement avec `<fieldset>` et `<legend>` (ex. « Coordonnées », « Choix de série »).

3. Utiliser `required` sur les champs obligatoires (validation HTML5 navigateur).
4. Prévoir un `name` explicite pour chaque contrôle (clé côté serveur).

## 8.6 Organisation d'un site, balises de section et limites du HTML

### MÉTHODE Organiser l'arborescence d'un site web statique

1. Créer un dossier racine (ex. `site_lycee__`).
2. Placer `index.html` (page d'accueil obligatoire) à la racine.
3. Créer des sous-dossiers thématiques : `images/`, `pages/`, `styles/` (pour CSS plus tard), `scripts/` (pour JS plus tard), `media/`.
4. Nommage strict : minuscules, pas d'espaces, pas d'accents, tirets bas `_` ou tirets `-` seulement (ex. `photo_classe.jpg`, `mentions-legales.html`).
5. Utiliser des chemins relatifs dans tous les liens (`href`, `src`).

### PROPRIÉTÉ Balises de structure sémantique (HTML5)

Ces balises n'ont pas d'affectation visuelle par défaut mais structurent le document pour le référencement (SEO), l'accessibilité et la maintenance.

- `<div>` : conteneur bloc générique (pas de sens sémantique). Utilisé pour le layout CSS.
- `<span>` : conteneur en ligne (inline) générique. Pour styliser une portion de texte.
- `<header>` : en-tête de page ou de section (logo, titre, navigation principale).
- `<nav>` : zone de navigation principale (menus).
- `<main>` : contenu principal unique de la page.
- `<section>` : section thématique du contenu (généralement avec un titre `<h1>`-`<h6>`).
- `<article>` : contenu autonome, syndiquable (article de blog, fiche produit).
- `<aside>` : contenu annexe (barre latérale, publicité, liens connexes).
- `<footer>` : pied de page (copyright, mentions légales, contacts).

Au programme de Première TI : `<div>`, `<section>`, `<span>` sont explicitement demandés. L'usage de `<header>`, `<nav>`, `<footer>` est une bonne pratique professionnelle fortement encouragée.

**À RETENIR** Limites du HTML (statique)

- Pas de traitement de données : un formulaire HTML seul ne peut pas enregistrer en base, envoyer un email, calculer une moyenne. Il faut un langage côté serveur (PHP, Python, Node.js) ou client (JavaScript) pour traiter l'événement `submit`.
- Pas de mise en page avancée : HTML décrit la structure, pas l'apparence (couleurs, polices, grille responsive, animations). C'est le rôle du CSS (leçon suivante).
- Pas d'interactivité dynamique : pas de modification du DOM sans rechargement, pas de validation instantanée, pas de carrousel, pas de carte interactive. C'est le rôle du JavaScript (leçon suivante).
- Contenu dupliqué : le menu, le pied de page, l'en-tête sont copiés-collés dans chaque page. Maintenance difficile. Les includes côté serveur (PHP `include`) ou les composants (JS frameworks) résolvent cela en Terminale.

**8.7** Activité d'intégration guidée : Mini-site du Lycée Bilingue de Nkol-Eton**EXPÉRIENCE** Réalisation d'un site vitrine statique (3 pages)

Durée estimée : 4 h (2 séances de 2 h). Matériel : Postes de travail (salle multimédia), navigateur (Firefox/Chrome), éditeur de code (VS Code / Notepad++), image `logo.png` fournie.

Objectifs : Mobiliser l'ensemble des savoir-faire de la leçon dans un projet concret : arborescence, structure HTML5, encodage, liens relatifs + infobulles, image cliquable, tableau avec attributs et fusions, listes imbriquées, balises de section, formulaire complet, test navigateur.

Situation : Le Lycée Bilingue de Nkol-Eton (région du Centre) prépare la rentrée. Le proviseur, M. Essomba, commande un site vitrine de 3 pages pour informer parents et nouveaux élèves. Chaque binôme de Première TI livre le site complet, testé, et présente son arborescence.

Données fournies :

- Page d'accueil (`index.html`) : Titre principal (nom du lycée), paragraphe avec accents (« établissement créé en 1992, situé à Nkol-Eton, accueillant 1 250 élèves »), logo `logo.png` cliquable (retour accueil), menu navigation 3 liens relatifs avec infobulles.
- Page « Résultats » (`pages/resultats.html`) : Tableau taux de réussite Probatoire TI 2024 (C : 78%, D : 71%, TI : 85%). Ligne titre fusionnée (`colspan`), cellule « 2024 » fusionnée verticalement (`rowspan=3`). Liste imbriquée des filières (Littéraire : A4 ; Scientifique : C, D ; Technique : TI, F4).
- Page « Inscription » (`pages/inscription.html`) : Formulaire pré-inscription : Nom (texte), Date naissance (date), Sexe (radio M/F), Série (liste déroulante A4, C, D, TI),

Activités (checkbox Football, Chorale, Club info), Boutons Envoyer / Effacer. Champs regroupés dans `<div>` ou `<section>`.

Consignes pas à pas :

1. **Arborescence** : Créer `site_lycee/` sur le Bureau. Y placer `index.html`. Créer `pages/` (`resultats.html`, `inscription.html`) et `images/` (`logo.png`). Dessiner l'arborescence sur papier.
2. **Structure de base** : Écrire le squelette HTML5 (`<!DOCTYPE html>`, `<html lang="fr">`, `<head>` avec `<meta charset="utf-8">` et `<title>`, `<body>`) dans les 3 fichiers. Pourquoi `meta charset` est-il vital ici?
3. **Page Accueil** : Insérer `<h1>`, paragraphe `<p>`, logo cliquable (`<a><img></a>`), menu `<nav>` avec 3 liens `<a href="...">` + `title`. Chemins vers `pages/resultats.html` et `pages/inscription.html`.
4. **Page Résultats** : Construire le `<table>` avec attributs `border`, `width`, `align`, `cellpadding`, `cellspacing`, `id`. Gérer `colspan="3"` sur `<th>` titre, `rowspan="3"` sur premier `<td>` de la 1ère ligne de données. Écrire la liste imbriquée `<ul><li>...<ul><li>...</li></ul></li></ul>`. Liens retour : `../index.html` et `inscription.html`.
5. **Page Inscription** : Construire `<form action="traitement.html" method="post">`. Un `<div>` par groupe de champs. `<label for="id">` + `<input id="id" name="...">`. Radios : même `name="sexe"`. Checkboxes : `name` distincts. `<select>` avec `<option value="TI" selected>`. Boutons `type="submit"` et `type="reset"`. Lien retour `../index.html`.
6. **Test & Débogage** : Ouvrir `index.html`. Vérifier : accents, logo cliquable, menu (infobulles, liens), tableau (fusions, alignement), formulaire (radio exclusif, checkbox multiples, liste, boutons). Corriger erreurs 404 (chemins), tableaux cassés (mauvais `rowspan`), radios non groupés.
7. **Présentation** : Montrer l'arborescence et le site fonctionnel. Citer une limite du HTML observée.

Compétences évaluées : Organiser, Coder (HTML), Tester, Communiquer (arborescence, limites).

**EXERCICE RÉSOLU 7****Corrigé complet commenté du mini-site**

Énoncé : Réaliser le site de trois pages décrit ci-dessus avec l'arborescence imposée.

**SOLUTION**

Étape 1 : Arborescence du site (organisation professionnelle).

```
site_lycee/  
  index.html  
  pages/  
    resultats.html  
    inscription.html  
  images/  
    logo.png
```

**Rappel :** Noms en minuscules, sans accents ni espaces (site\_lycee, pas Site Lycée). Les serveurs Linux (majorité du web) sont sensibles à la casse.

**Étape 2 :** Page d'accueil (index.html) – Structure, encodage, logo cliquable, menu.

```
<!DOCTYPE html>  
<html lang="fr">  
  <head>  
    <meta charset="utf-8">  
    <title>Lycée Bilingue de Nkol-Eton – Accueil</title>  
  </head>  
  <body>  
    <header>  
      <!-- Logo cliquable : lien vers l'accueil (même page) -->  
      <a href="index.html" title="Retour à la page d'accueil">  
          
      </a>  
      <h1>Lycée Bilingue de Nkol-Eton</h1>  
    </header>  
  
    <nav>  
      <!-- Liens relatifs vers le sous-dossier pages/ -->  
      <a href="index.html" title="Page d'accueil">Accueil</a> |  
      <a href="pages/resultats.html" title="Consulter les taux de réussite au Probatoire">Résultats</a> |  
      <a href="pages/inscription.html" title="Accéder au formulaire de pré-inscription">Inscription</a>  
    </nav>  
  
    <main>  
      <section id="presentation">  
        <h2>Bienvenue</h2>  
        <p>Établissement créé en 1992, situé à Nkol-Eton,  
        notre lycée accueille <span>1 250 élèves</span>  
        de la Seconde à la Terminale.</p>  
      </section>  
    </main>  
  
    <footer>  
      <p>&copy; 2025 Lycée Bilingue de Nkol-Eton – Tous droits réservés</p>  
    </footer>  
  </body>  
</html>
```

**Points clés :** <meta charset="utf-8"> gère les accents (« établissement », « élèves »). Logo cliquable = <img> dans <a>. Menu : chemins pages/fichier.html (descente). Balises sémantiques <header>, <nav>, <main>, <section>, <footer>, <span>.

### Étape 3 : Page Résultats (pages/resultats.html) – Tableau à fusions, liste imbriquée, liens retour.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Résultats du Probatoire – Lycée Bilingue de Nkol-Eton</title>
</head>
<body>
  <header>
    <a href="../index.html" title="Retour à l'accueil">
      
    </a>
    <h1>Résultats du Lycée Bilingue de Nkol-Eton</h1>
  </header>

  <main>
    <section id="tableau-resultats">
      <table border="1" width="70%" align="center"
        cellpadding="8" cellspacing="0" id="tab-probatoire">
        <!-- Ligne 1 : Titre fusionné sur 3 colonnes -->
        <tr>
          <th colspan="3">Résultats du Probatoire 2024</th>
        </tr>
        <!-- Ligne 2 : En-têtes de colonnes -->
        <tr>
          <th>Année</th>
          <th>Série</th>
          <th>Taux de réussite</th>
        </tr>
        <!-- Ligne 3 : 1ère donnée + cellule '2024' rowspan=3 -->
        <tr>
          <td rowspan="3" align="center">2024</td>
          <td align="center">C</td>
          <td align="center">78 %</td>
        </tr>
        <!-- Ligne 4 : 2ème donnée (pas de cellule Année, héritée du rowspan) -->
        <tr>
          <td align="center">D</td>
          <td align="center">71 %</td>
        </tr>
        <!-- Ligne 5 : 3ème donnée -->
        <tr>
          <td align="center">Tl</td>
          <td align="center">85 %</td>
        </tr>
      </table>
    </section>

    <section id="filieres">
      <h2>Nos filières de formation</h2>
      <ul>
        <li>Littéraire
          <ul>
            <li>A4</li>
          </ul>
        </li>
        <li>Scientifique
      </ul>
    </section>
  </main>
</body>
</html>
```

```

        <ul>
          <li>C</li>
          <li>D</li>
        </ul>
      </li>
      <li>Technique
        <ul>
          <li>T1</li>
          <li>F4</li>
        </ul>
      </li>
    </ul>
  </section>
</main>

<nav>
  <!-- Remonter d'un niveau avec ../ -->
  <a href="../index.html" title="Retour à l'accueil">Accueil</a> |
  <!-- Même dossier : pas de ../ -->
  <a href="inscription.html" title="S'inscrire">Inscription</a>
</nav>
</body>
</html>

```

#### Analyse du tableau :

- Ligne 1 (<tr>) : 1 seule cellule <th colspan="3"> → 3 colonnes fusionnées.
- Ligne 2 : 3 cellules <th> normales.
- Ligne 3 : 3 cellules écrites (<td rowspan="3">, <td>, <td>). La 1ère s'étale sur 3 lignes.
- Ligne 4 : 2 cellules écrites (<td>, <td>). La 1ère colonne est occupée par le rowspan de la ligne 3.
- Ligne 5 : 2 cellules écrites. Idem.

Liens : Depuis pages/, ../index.html remonte à la racine. inscription.html est dans le même dossier.

Étape 4 : Page Inscription (pages/inscription.html) – Formulaire complet, accessibilité, section.

```

<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Pré-inscription – Lycée Bilingue de Nkol-Eton</title>
</head>
<body>
  <header>
    <a href="../index.html" title="Retour à l'accueil">
      
    </a>
    <h1>Formulaire de pré-inscription</h1>
  </header>

  <main>

```

```
<form action="traitement.html" method="post">
  <!-- Champ texte : Nom -->
  <div>
    <label for="nom">Nom de l'élève :</label>
    <input type="text" id="nom" name="nom" size="30"
      placeholder="Ex : Essomba" required>
  </div>

  <!-- Champ date : Naissance -->
  <div>
    <label for="naissance">Date de naissance :</label>
    <input type="date" id="naissance" name="naissance" required>
  </div>

  <!-- Boutons radio : Sexe (même name = groupe exclusif) -->
  <div>
    <fieldset>
      <legend>Sexe :</legend>
      <label>
        <input type="radio" name="sexe" value="M" id="sexe_m" required>
        Masculin
      </label>
      <label>
        <input type="radio" name="sexe" value="F" id="sexe_f">
        Féminin
      </label>
    </fieldset>
  </div>

  <!-- Liste déroulante : Série -->
  <div>
    <label for="serie">Série demandée :</label>
    <select id="serie" name="serie" required>
      <option value="">-- Choisir --</option>
      <option value="A4">A4 (Littéraire)</option>
      <option value="C">C (Scientifique)</option>
      <option value="D">D (Scientifique)</option>
      <option value="TI" selected>TI (Technologique)</option>
    </select>
  </div>

  <!-- Cases à cocher : Activités (names distincts = choix multiples) -->
  <div>
    <fieldset>
      <legend>Activités parascolaires :</legend>
      <label>
        <input type="checkbox" name="foot" value="1" id="act_foot">
        Football
      </label>
      <label>
        <input type="checkbox" name="chorale" value="1" id="act_chorale">
        Chorale
      </label>
      <label>
        <input type="checkbox" name="club_info" value="1" id="act_club">
        Club informatique
      </label>
    </fieldset>
  </div>
```

```
        </fieldset>
    </div>

    <!-- Boutons d'action -->
    <div>
        <input type="submit" value="Envoyer">
        <input type="reset" value="Effacer">
    </div>
</form>
</main>

<footer>
    <p><a href="../index.html" title="Retour à l'accueil">Retour à l'accueil</a></p>
</footer>
</body>
</html>
```

#### Points clés :

- `<label for="id"> + <input id="id">` : **accessibilité** (clic sur le texte = focus).
- **Radios** : `name="sexe"` identique → un seul choix possible. `value` différent (M/F).
- **Checkboxes** : `name` différents (foot, chorale, club\_info) → choix multiples indépendants.
- `required` : validation HTML5 navigateur (empêche envoi si vide).
- `<fieldset>/<legend>` : regroupement sémantique et visuel (bordure par défaut).
- `action="traitement.html"` : fictif ici (limite HTML statique).

Étape 5 : Test et validation. Ouvrir `site_lycee/index.html` dans le navigateur.

- ≡ Accents corrects (encodage UTF-8).
- ≡ Logo cliquable (retour accueil), infobulles au survol (`title`).
- ≡ Menu : liens vers `pages/resultats.html` et `pages/inscription.html` fonctionnent.
- ≡ Page Résultats : tableau centré, bordures visibles, titre fusionné, cellule « 2024 » verticale, liste imbriquée correcte.
- ≡ Page Inscription : formulaire saisissable, radio exclusif, checkbox multiples, liste TI pré-sélectionnée, boutons Envoyer/Effacer actifs.
- ≡ Liens retour (`../index.html`) fonctionnent depuis `pages/`.
- ≡ Clic sur « Envoyer » → page `traitement.html` introuvable (normal) ou affichage brut des données en URL si `method="get"`. Preuve de la limite du HTML statique.

#### ATTENTION Erreurs fréquentes au Probatoire et en TP – Check-list de révision

- Chemins relatifs : Oublier `../` pour remonter depuis `pages/` vers `index.html` ou `images/`.

→ Erreur 404 « Fichier introuvable ».

- Encodage : Absence de `<meta charset="utf-8">` → « Établissement », « Éèves ».
- Nommage fichiers : Espaces (Page Résultats.html), accents (inscription.html), majuscules → Liens cassés sur serveur Linux.
- Tableau & fusions : Compter les cellules après fusion. Ex : `rowspan="3"` sur 3 lignes → les 2 lignes suivantes ont une cellule de moins. Ne pas écrire le `<td>` fantôme.
- Radios : `name` différents → on peut cocher Masculin ET Féminin en même temps.
- Checkboxes : `name` identiques → un seul choix possible (comportement radio).
- Formulaire : Oublier `name` sur un champ → donnée non envoyée au serveur.
- Accessibilité : `alt` manquant sur image, `<label>` non lié (`for/id`).
- Imbrication : Fermer les balises dans l'ordre inverse (LIFO) : `<ul><li><ul><li></li></ul></li></ul>`. Ne pas croiser : `<a><b></a></b>` (invalide).

**Bilan de la leçon.** Cette leçon a couvert l'intégralité du module « Programmation Web statique en HTML » du programme officiel de Première TI. Tu sais maintenant :

1. Structurer une page HTML5 valide avec `<!DOCTYPE html>`, `<meta charset="utf-8">`, `<head>`, `<body>`.
2. Organiser une **arborescence** de site propre (dossiers images/, pages/, noms normalisés).
3. Maîtriser les **liens relatifs** (dossier/, ../) et les **infobulles** (title).
4. Insérer des **médias** : images (`<img>`), images cliquables (`<a><img></a>`), sons (`<audio>`), vidéos (`<video>`).
5. Construire des **tableaux** complexes avec attributs de présentation (border, width, align, cellpadding, cellspacing) et **fusions de cellules** (colspan, rowspan).
6. Imbriquer des **listes** (`<ul>/<ol>` dans `<li>`).
7. Réaliser des **formulaires** accessibles et complets : `<form>`, `<input>` (text, date, radio, checkbox, submit, reset), `<select>/<option>`, `<label>`, `<fieldset>`.
8. Structurer le contenu avec les balises de **section** (`<div>`, `<section>`, `<span>`, `<header>`, `<nav>`, `<footer>`).
9. Identifier les **limites du HTML** : langage de **balisage** (structure + contenu), **statique**, sans traitement de données, sans mise en page avancée, sans interactivité dynamique → nécessite **CSS** (style) et **JavaScript** (comportement) côté client, puis langages serveur (PHP, Python...) côté serveur (Terminale).

Félicitations ! Tu as posé la première pierre du développement web. La prochaine leçon (CSS) va transformer ce squelette en une interface visuelle professionnelle.

## 9 MÉTHODES ET EXERCICES RÉSOLUS

### 9.1 Fiches méthodes et exercices résolus

#### MÉTHODE Afficher correctement les caractères accentués

Pour que les accents (é, è, à, ç, ô...) s'affichent correctement dans tous les navigateurs, deux conditions indissociables doivent être respectées :

1. Encodage du fichier : Enregistrer le fichier impérativement en UTF-8 (menu Encodage → Convertir en UTF-8 dans Notepad++ ou VS Code). C'est l'encodage universel du web.
2. Déclaration dans le code : Placer en tout premier dans la balise `<head>` la balise méta :  
`<meta charset="utf-8">`.

**Réflexe Bac :** La balise `<meta charset="utf-8">` est orpheline (pas de balise fermante). Elle doit être la première enfant de `<head>`, avant `<title>`, pour que le navigateur lise le titre lui-même avec le bon encodage. Si des symboles type Ã© ou ¢ apparaissent : l'encodage du fichier (disque) ne correspond pas au `charset` déclaré (navigateur).

#### EXERCICE RÉSOLU 8 Une page qui affiche mal les accents

Le site du lycée de Bafoussam affiche « Bienvenue au LycÃ©e » au lieu de « Bienvenue au Lycée ». Le code source commence ainsi :

```
<html>
<head>
<title>Lycee de Bafoussam</title>
</head>
<body>Bienvenue au Lycee</body>
</html>
```

1) Identifier la cause du problème. 2) Corriger le code en respectant la structure HTML5 complète.

#### SOLUTION

1) Cause. L'affichage « LycÃ©e » est le symptôme classique d'un fichier enregistré en UTF-8 mais interprété par le navigateur en ISO-8859-1 (Latin-1), car aucun `charset` n'est déclaré dans le `<head>`. Le navigateur « devine » mal. De plus, la déclaration de type de document (`<!DOCTYPE html>`) est absente.

2) Correction. On ajoute le `DOCTYPE HTML5`, la balise méta `charset` en première position dans `<head>`, et on s'assure que le fichier est bien enregistré en UTF-8 (sans BOM de préférence) :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Lycée de Bafoussam</title>
</head>
<body>
  <p>Bienvenue au Lycée</p>
</body>
</html>
```

**Conclusion :** La déclaration `<meta charset="utf-8">` dans `<head>`, associée à un enregistrement du fichier en UTF-8, garantit l’affichage correct des caractères accentués. L’attribut `lang="fr"` sur `<html>` aide aussi l’accessibilité et le référencement.

#### MÉTHODE Imbriquer des listes et insérer un hyperlien avec infobulle

Listes imbriquées (règle d’or : l’imbrication se fait dans un `<li>`) :

1. Une liste non ordonnée `<ul>` (puces) ou ordonnée `<ol>` (numéros) contient des items `<li>`.
2. Pour créer une sous-liste, on ouvre la nouvelle liste (`<ul>` ou `<ol>`) à l’intérieur d’un `<li>` parent, avant de fermer ce `<li>`.
3. On ferme les balises dans l’ordre inverse de l’ouverture (LIFO) : `</ol>` puis `</li>` puis `</ul>`.

Hyperlien (`<a>`) avec infobulle :

1. Syntaxe : `<a href="URL" title="Texte de l'infobulle">Contenu cliquable</a>`.
2. L’attribut `href` (Hypertext REFerence) reçoit une URL :
  - Absolue : Adresse complète avec protocole (`https://www.minesec.gov.cm`, `mailto:contact@ecole.cm`).
  - Relative : Chemin depuis la page courante (`contact.html`, `../images/logo.png`, `/pages/accueil.html`).
3. L’attribut `title` affiche une infobulle (tooltip) au survol de la souris (accessibilité).
4. Le contenu cliquable peut être du texte, une image, ou un bloc (HTML5).

#### EXERCICE RÉSOLU 9 Menu du site d’une bibliothèque scolaire

On veut créer le menu suivant : une liste à puces « Nos services » contenant « Prêt de livres » (avec une sous-liste numérotée : « Romans », « Manuels scolaires », « Revues ») et « Salle multimédia » ; puis un lien externe vers le site du MINESEC avec l’infobulle « Site of-

ficiel du Ministère ». Rédiger le code HTML complet et valide.

**SOLUTION**

Raisonnement. La sous-liste numérotée (<ol>) doit être placée à l'intérieur du <li> « Prêt de livres ». Le lien vers le MINESEC est externe : on utilise une URL absolue (HTTPS de préférence), et l'infobulle est donnée par l'attribut `title`.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Bibliothèque du Lycée – Menu</title>
</head>
<body>
  <nav>
    <ul>
      <li>Prêt de livres
        <ol>
          <li>Romans</li>
          <li>Manuels scolaires</li>
          <li>Revue</li>
        </ol>
      </li>
      <li>Salle multimédia</li>
    </ul>

    <p>
      <a href="https://www.minesec.gov.cm" title="Site officiel du Ministère" target="_blank" rel="noopener">
        Visiter le site du MINESEC
      </a>
    </p>
  </nav>
</body>
</html>
```

Vérification : Chaque <li> ouvert est fermé ; la sous-liste <ol> est bien dans le <li> parent (pas entre deux <li>) ; le lien absolu commence par `https://` ; `target="_blank"` ouvre dans un nouvel onglet (bonnes pratiques : ajouter `rel="noopener"` pour la sécurité).

Conclusion : L'imbrication correcte des listes (<ul>/<ol>/<li>) et le couple d'attributs `href/title` répondent exactement au cahier des charges.

**MÉTHODE****Insérer un média : image, image cliquable, son, vidéo**

1. Image (<img>) : .

- **src (source)** : URL relative ou absolue du fichier image (JPG, PNG, WebP, SVG, GIF).
- **alt (alternative)** : Obligatoire (accessibilité lecteurs d'écran, référencement, af-

fichage si image absente).

- `width/height` : Dimensions en pixels (réserve l'espace, évite le « saut » de mise en page au chargement).

`<img>` est une **balise orpheline** (auto-fermante en HTML5 : `<img ...>`).

2. Image cliquable : On imbrique `<img>` à l'intérieur d'un lien `<a href="...">...</a>`.
3. Son (`<audio>`) : `<audio src="URL" controls></audio>`.
  - `controls` : Affiche les boutons lecture/pause/volume (interface navigateur).
  - Formats courants : MP3, OGG, WAV. Préférer `<source>` multiples pour compatibilité.
4. Vidéo (`<video>`) : `<video src="URL" controls width="..."></video>`.
  - `controls, width, height, poster` (image d'attente).
  - Formats : MP4 (H.264) universel, WebM. Utiliser `<source>` pour alternatives.
5. Vérification cruciale : Le chemin (URL) du média doit être correct relativement à l'emplacement du fichier HTML (pas relativement au dossier racine du serveur).

**Rappel** : `<img>` orpheline ; `<audio>` et `<video>` ont une balise fermante (permettent du contenu de repli entre les balises).

#### EXERCICE RÉSOLU 10 Page d'accueil d'un cybercafé

La page d'accueil (`index.html` à la racine) du cybercafé « NetExpress » de Douala doit afficher : le logo `logo.png` (dans `images/`) qui, cliqué, renvoie vers `services.html` ; puis une vidéo de présentation `promo.mp4` (dans `video/`) avec contrôles. Rédiger le code HTML5 complet.

#### SOLUTION

Raisonnement. Page à la racine ⇒ URL relatives : `images/logo.png` et `video/promo.mp4` (descente dans sous-dossiers). Image cliquable = `<a>` autour de `<img>`. Vidéo = `<video controls>`.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>NetExpress – Cybercafé Douala</title>
</head>
<body>
  <header>
    <!-- Image cliquable : le lien entoure l'image -->
    <a href="services.html" title="Nos services">
      
  </header>
  <div>
    <video src="video/promo.mp4" controls>
  </div>
</body>
</html>
```

```

        width="200" height="120">
    </a>
</header>

<main>
    <h1>Bienvenue chez NetExpress</h1>
    <!-- Vidéo avec boutons de lecture natifs -->
    <video src="video/promo.mp4" controls width="640" poster="images/affiche.jpg">
        <p>Votre navigateur ne supporte pas la vidéo HTML5.
        <a href="video/promo.mp4">Télécharger la vidéo</a>.</p>
    </video>
</main>
</body>
</html>

```

**Conclusion :** L'imbrication `<a> > <img>` crée l'image cliquable; la balise `<video>` avec `controls` et contenu de repli assure la lecture via des URL relatives correctes. L'attribut `poster` améliore l'expérience utilisateur avant lecture.

#### MÉTHODE Construire un tableau et fusionner des cellules

1. Structure de base : `<table>` → lignes `<tr>` → cellules `<td>` (données) ou `<th>` (en-têtes, gras + centrées par défaut).
2. Attributs de `<table>` (HTML4/transitionnel, encore au programme) : `border="1"` (bordure), `width="80%"` (largeur), `cellpadding="5"` (marge intérieure), `cellspacing="2"` (espace entre cellules), `align="center"` (centrage tableau), `id="identifiant"` (unique pour CSS/JS). Note : En HTML5 moderne, on privilégie CSS pour la présentation (`border-collapse`, `padding`, `margin`).
3. Fusion horizontale (`colspan`) : Dans un `<td>` ou `<th>`, `colspan="n"` fusionne  $n$  cellules sur la même ligne. On supprime alors les  $n - 1$  cellules `<td>` suivantes de cette ligne.
4. Fusion verticale (`rowspan`) : `rowspan="n"` fusionne  $n$  cellules sur la même colonne. On supprime la cellule correspondante dans chacune des  $n - 1$  lignes suivantes.
5. Vérification systématique (règle de cohérence) : Chaque ligne `<tr>` doit occuper le même nombre total de colonnes en comptant : cellules normales + valeurs `colspan` + cellules « retenues » par un `rowspan` venu du dessus.

#### EXERCICE RÉSOLU 11 Tableau des notes d'une classe de Première TI

Créer un tableau HTML5 (avec bordure visible, largeur 80%, centré) comportant :

- Une ligne d'en-tête : « Matière | Note | Appréciation ».
- Une ligne de données : « Algorithmique | 17 | Très bien ».

- Une ligne de données : « Web Statique | 15 | Bien ».
- Une dernière ligne fusionnant les 3 colonnes : « Moyenne générale : 15,5 ».

Utiliser `cellpadding="5"` et `cellspacing="0"`.

#### SOLUTION

Raisonnement. Trois colonnes  $\Rightarrow$  3 cellules par ligne (sauf fusion). Dernière ligne : fusion horizontale des 3 colonnes  $\Rightarrow$  un seul `<td colspan="3">`, aucune autre cellule sur cette ligne.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Bulletin de notes – 1ère TI</title>
</head>
<body>
  <table border="1" width="80%" cellpadding="5" cellspacing="0" align="center" id="bulletin">
    <caption>Résultats du 1er trimestre – Élève : Mbarga Jean</caption>
    <tr>
      <th>Matière</th>
      <th>Note</th>
      <th>Appréciation</th>
    </tr>
    <tr>
      <td>Algorithmique</td>
      <td>17</td>
      <td>Très bien</td>
    </tr>
    <tr>
      <td>Web Statique</td>
      <td>15</td>
      <td>Bien</td>
    </tr>
    <tr>
      <td colspan="3" style="text-align:center;font-weight:bold;">Moyenne générale : 15,5</td>
    </tr>
  </table>
</body>
</html>
```

Vérification : Ligne 1 (en-têtes) : 3 `<th>` = 3 colonnes. Ligne 2 : 3 `<td>` = 3 colonnes. Ligne 3 : 3 `<td>` = 3 colonnes. Ligne 4 : 1 `<td colspan="3">` = 3 colonnes. Tableau équilibré.

Conclusion : L'attribut `colspan` permet la fusion horizontale ; on retire scrupuleusement les cellules absorbées pour conserver un tableau régulier. L'ajout de `<caption>` (légende) et d'un peu de CSS inline (`style`) améliore la lisibilité.

#### MÉTHODE

#### Créer un formulaire HTML

1. **Conteneur** : `<form action="URL_traitement" method="get|post">...</form>`.
  - **action** : URL du script serveur qui traitera les données (PHP, Python, Node.js... vu en Terminale). En attendant, on met souvent `action="#"` ou `action=""` (page courante) pour tester la structure.
  - **method** : `get` (données dans l'URL, pour recherche/filtrage) ou `post` (données dans le corps HTTP, pour inscription/modification, plus sécurisé).
2. **Champ de saisie texte** : `<input type="text" name="identifiant" id="identifiant" placeholder="Votre nom">`. `type="password"` masque la saisie. **name** est indispensable pour l'envoi des données.
3. **Boutons radio (choix unique)** : `<input type="radio" name="groupe" value="valeur" id="id1">`. Règle stricte : Tous les boutons d'un même choix exclusif portent le même **name**.
4. **Cases à cocher (choix multiples)** : `<input type="checkbox" name="choix[]" value="val" id="id2">`. **name** peut être identique (tableau côté serveur) ou différent. **value** = valeur envoyée si cochée.
5. **Liste déroulante** : `<select name="ville" id="ville">` contenant `<option value="code">Libellé</option>`.
6. **Boutons d'action** : `<input type="submit" value="Envoyer">` (envoie le formulaire), `<input type="reset" value="Annuler">` (remet à zéro), `<button type="button">Action JS</button>` (bouton neutre pour JavaScript).
7. **Accessibilité (Obligatoire au Bac)** : Associer explicitement chaque champ à son étiquette via `<label for="id_du_champ">Libellé</label>` et `id="id_du_champ"` sur le `<input>/<select>/<textarea>`. Cela agrandit la zone cliquable et aide les lecteurs d'écran.

**EXERCICE RÉSOLU 12****Formulaire d'inscription à un concours d'informatique**

Créer le formulaire d'inscription au concours régional : Nom complet (texte), Sexe (radio : Masculin/Féminin), Ville (liste : Yaoundé, Douala, Bafoussam, Garoua), Épreuves choisies (cases à cocher : Algorithmique, Web, Maintenance, Réseaux), Bouton « Valider mon inscription ». Utiliser l'association `label for / id`.

**SOLUTION**

Raisonnement.

- Choix unique (Sexe)  $\Rightarrow$  `type="radio"` même `name="sexe"`.
- Choix multiple (Épreuves)  $\Rightarrow$  `type="checkbox"` (noms distincts ou tableau `epreuves[]`).
- Choix unique dans liste (Ville)  $\Rightarrow$  `<select>`.

- `action="#"` pour démo statique ; `method="post"` car inscription (modification état serveur).

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Inscription Concours Informatique</title>
</head>
<body>
  <form action="#" method="post" id="form-inscription">
    <fieldset>
      <legend>Informations candidat</legend>

      <div>
        <label for="nom">Nom complet :</label>
        <input type="text" name="nom" id="nom" placeholder="Ex : Mbarga Jean" required>
      </div>

      <div>
        <label>Sexe :</label>
        <label for="sexe-m"><input type="radio" name="sexe" value="M" id="sexe-m" required>
Masculin</label>
        <label for="sexe-f"><input type="radio" name="sexe" value="F" id="sexe-f"> Féminin</
label>
      </div>

      <div>
        <label for="ville">Ville de résidence :</label>
        <select name="ville" id="ville" required>
          <option value="">-- Choisir --</option>
          <option value="yde">Yaoundé</option>
          <option value="dla">Douala</option>
          <option value="baf">Bafoussam</option>
          <option value="grs">Garoua</option>
        </select>
      </div>

      <div>
        <label>Épreuves choisies :</label>
        <label for="ep-algo"><input type="checkbox" name="epreuves[]" value="algo" id="ep-
algo"> Algorithmique</label>
        <label for="ep-web"><input type="checkbox" name="epreuves[]" value="web" id="ep-
web"> Web Statique</label>
        <label for="ep-maint"><input type="checkbox" name="epreuves[]" value="maint" id="ep-
-maint"> Maintenance</label>
        <label for="ep-res"><input type="checkbox" name="epreuves[]" value="res" id="ep-res
"> Réseaux</label>
      </div>

      <div>
        <input type="submit" value="Valider mon inscription">
        <input type="reset" value="Effacer le formulaire">
      </div>
    </fieldset>
  </form>
</body>
```

| `</html>`

**Conclusion :** Chaque type de champ (`input`, `select`, `textarea`) correspond à un besoin précis (texte, choix unique, choix multiple, liste). L'association `<label for> / id` est la norme d'accessibilité. Le regroupement par `<fieldset>/<legend>` structure le formulaire. Le `name="epreuves[]"` (avec crochets) prépare la réception côté serveur (tableau PHP).

**MÉTHODE****Organiser les dossiers d'un site et structurer avec les balises de section****1. Arborescence type (racine du site) :**

```
mon_site/
  index.html          (page d'accueil, point d'entrée)
  css/
    style.css
  js/
    script.js
  images/
    logo.png
    banniere.jpg
  pages/
    contact.html
    services.html
```

**2. URL relatives (navigation dans l'arborescence) :**

- **Descendre** : `images/logo.png`, `pages/contact.html`.
- **Remonter** : `../index.html` (depuis `pages/`), `../../fichier` (deux niveaux).
- **Racine du site (absolu depuis racine)** : `/images/logo.png` (début par `/`).

**3. Balises de structure sémantique (HTML5) :**

- `<header>` : En-tête de page ou de section (logo, titre, nav principal).
- `<nav>` : Navigation principale (menus).
- `<main>` : Contenu principal unique de la page.
- `<section>` : Regroupement thématique avec un titre (ex : « Actualités », « Services »).
- `<article>` : Contenu autonome, syndiquable (ex : article de blog, fiche produit).
- `<aside>` : Contenu annexe (barre latérale, pub, liens connexes).
- `<footer>` : Pied de page (copyright, mentions légales, contacts).

- `<div>` : Conteneur générique de bloc (sans sens sémantique), pour mise en page CSS (grilles, flexbox).
  - `<span>` : Conteneur générique en ligne (pas de retour à la ligne), pour styliser un morceau de phrase.
4. Identification pour CSS/JS : Attribut `id` (unique dans la page, ex : `id="menu-principal"`) ou `class` (réutilisable, ex : `class="carte-service"`).
  5. Limites du HTML (à connaître et énoncer) :
    - (a) HTML est un langage de balisage (structure/contenu), pas un langage de programmation : pas de variables, boucles, conditions, calculs, fonctions.
    - (b) HTML produit des pages statiques : même contenu pour tous, pas d'interaction avec une base de données, pas de session utilisateur, pas de traitement de formulaire (nécessite JS côté client + langage serveur côté serveur).
    - (c) Le rendu visuel dépend du navigateur (styles par défaut différents) ⇒ nécessité du CSS.
    - (d) Accessibilité et SEO limités sans attributs sémantiques (`alt`, `label`, `structure` `<h1>` - `<h6>`, balises HTML5).

**EXERCICE RÉSOLU 13 Structurer la page d'accueil du site du lycée**

Le site du lycée possède l'arborescence : racine (`index.html`), `images/`, `css/`, `pages/`. On demande : 1) Depuis `pages/contact.html`, écrire la balise pour afficher `images/batiment.jpg`. 2) Structurer `index.html` en zones sémantiques : en-tête (logo + titre), navigation (menu), contenu principal (section « Bienvenue » + section « Actualités »), pied de page. Mettre en évidence le mot « excellence » dans une phrase. 3) Citer deux limites majeures du HTML.

**SOLUTION**

1) Image depuis un sous-dossier `pages/`. Il faut remonter d'un niveau (`../`) pour atteindre la racine, puis redescendre dans `images/` :

```
<!-- Dans pages/contact.html -->

```

2) Structuration sémantique de `index.html`. On utilise les balises HTML5 (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`) pour le sens, et `<div>` uniquement si besoin de conteneur neutre pour le CSS. `<span>` pour le mot en ligne.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
```

```
<title>Lycée de Bafoussam – Accueil</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header id="entete">
    
    <h1>Lycée de Bafoussam</h1>
  </header>

  <nav id="menu-principal">
    <ul>
      <li><a href="index.html">Accueil</a></li>
      <li><a href="pages/presentation.html">Présentation</a></li>
      <li><a href="pages/contact.html">Contact</a></li>
    </ul>
  </nav>

  <main id="contenu-principal">
    <section id="bienvenue">
      <h2>Bienvenue</h2>
      <p>Un établissement d'<span class="mot-cle">excellence</span> au service de la réussite de
tous.</p>
    </section>

    <section id="actualites">
      <h2>Actualités</h2>
      <article>
        <h3>Rentrée 2024-2025</h3>
        <p>Les inscriptions sont ouvertes...</p>
      </article>
    </section>
  </main>

  <footer id="pied-page">
    <p>Contact : <a href="mailto:lycee.bafoussam@education.cm">lycee.bafoussam@education.
cm</a></p>
    <p>&copy; 2025 Lycée de Bafoussam – Tous droits réservés.</p>
  </footer>
</body>
</html>
```

**Note :** `<span class="mot-cle">` n'introduit pas de rupture de ligne (élément inline), contrairement à `<div>` (bloc). C'est le bon choix pour styliser un mot dans une phrase. Les `id` uniques (entete, menu-principal...) et `class` réutilisables (mot-cle) préparent le ciblage CSS.

### 3) Limites majeures du HTML.

1. **Statisme & Absence de logique :** HTML ne permet ni calculs, ni conditions (`if/else`), ni boucles, ni stockage de données. Une page HTML est un document figé servi tel quel.
2. **Aucune interaction serveur / Base de données :** HTML seul ne peut pas traiter un formulaire (enregistrer un élève), afficher des notes depuis une BDD, ni gérer une session utilisateur (connexion). Cela nécessite JavaScript (côté client) et un langage serveur (PHP, Python, Node.js) + SGBD (vus en Terminale).

**Conclusion :** Une arborescence claire, la maîtrise des URL relatives (../) et l'usage des balises sémantiques HTML5 (<header>, <nav>, <main>, <section>, <footer>, <span>) donnent un site bien organisé, accessible et maintenable, tout en gardant à l'esprit que HTML seul reste limité à la structure et au contenu statique.

#### ATTENTION Les 5 erreurs qui coûtent des points au Probatoire

1. Oublier `<meta charset="utf-8">` ou le placer hors du `<head>` (ex : dans `<body>`) : les accents s'affichent mal (Ã©, ) ⇒ sanction systématique. Astuce : Le taper en tout premier dans le `<head>`.
2. Fusionner des cellules sans supprimer les cellules absorbées : Avec `colspan="3"`, on n'écrit qu'une seule cellule `<td>` sur la ligne ; avec `rowspan="2"`, on supprime la cellule correspondante dans la ligne suivante. Sinon : colonnes décalées, tableau « cassé ».
3. Donner des `name` différents aux boutons radio d'un même groupe : L'utilisateur peut alors tout cocher (choix multiple au lieu d'unique). Même groupe ⇒ Même `name`.
4. Erreur d'URL relative (chemin cassé) : Depuis `pages/contact.html`, écrire `src="images/logo.png"` au lieu de `src="../images/logo.png"`. Le navigateur cherche `pages/images/logo.png` (inexistant) ⇒ image cassée (icône déchirée). Réflexe : « Suis-je dans un sous-dossier ? ⇒ ../ pour sortir. »
5. Confondre balises de structure et leurs rôles :
  - Utiliser `<div>` pour un mot dans une phrase (crée un bloc, retour à la ligne) au lieu de `<span>` (en ligne).
  - Oublier l'attribut `alt` sur `<img>` (accessibilité, SEO, validation W3C).
  - Écrire `<img>...</img>` ou `<br></br>` : ce sont des balises orphelines (void elements), syntaxe correcte : `<img ...>`, `<br>`.
  - Imbriquer une liste `<ul>` entre deux `<li>` au lieu de dans le `<li>` parent.

#### LESAVAIS-TU ? Le saviez-vous ? : Le premier site web au monde

Le tout premier site web de l'histoire a été mis en ligne le 20 décembre 1990 par Tim Berners-Lee au CERN (Suisse). Il tournait sur un ordinateur NeXT et expliquait le projet « WorldWideWeb » : comment créer des pages, des liens, et configurer un serveur. L'adresse originale `http://info.cern.ch/` est toujours accessible (restaurée en 2013). Il ne contenait que du HTML (pas de CSS, pas d'images, pas de JS) : du texte structuré avec des titres, des listes et des hyperliens. C'est la preuve que le cœur du web, c'est d'abord la structure de l'information et le lien hypertexte — exactement ce que vous apprenez en ce moment en Première TI !

**EXERCICE 7** Entraînement : Page « Profil Étudiant »

Créer le fichier `profil.html` (dans dossier `pages/`) pour un étudiant de 1ère TI. La page doit contenir :

1. Structure HTML5 complète (DOCTYPE, lang="fr", meta charset, title).
2. En-tête (<header>) : Photo de l'étudiant (../images/photo.jpg, cliquable vers ../index.html), Nom, Prénom, Classe.
3. Navigation (<nav>) : Liste non ordonnée de liens vers ../index.html (Accueil), notes.html (Mes notes), emploi-temps.html (Emploi du temps).
4. Contenu principal (<main>) avec deux sections :
  - « Informations personnelles » : Tableau (bordure, cellpadding="5") : Ville, Date de naissance, Email, Téléphone. Une ligne fusionnée sur 2 colonnes : « Représentant légal : M. Mbarga ».
  - « Compétences TI » : Liste à puces imbriquée : « Algorithmique » (sous-liste : « Tableaux », « Tris », « Recherche »), « Web Statique » (sous-liste : « HTML », « CSS »), « Maintenance » (sous-liste : « Matériel », « OS », « Réseau »).
5. Pied de page (<footer>) : Copyright + lien `mailto:` vers l'email de l'étudiant.
6. Un mot mis en évidence dans une phrase avec `<span class="highlight">`.

Indication : Attention aux chemins relatifs depuis `pages/` !

**CORRIGÉ** Exercice : Page « Profil Étudiant »

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Profil Étudiant – Mbarga Jean (1ère TI)</title>
  <link rel="stylesheet" href="../css/style.css">
</head>
<body>
  <header>
    <!-- Image cliquable vers l'accueil (racine) -->
    <a href="../index.html" title="Retour à l'accueil">
      
    </a>
    <h1>Mbarga Jean</h1>
    <p>Classe : <span class="highlight">1ère TI</span></p>
  </header>

  <nav>
    <ul>
      <li><a href="../index.html">Accueil</a></li>
```

```

        <li><a href="notes.html">Mes notes</a></li>
        <li><a href="emploi-temps.html">Emploi du temps</a></li>
    </ul>
</nav>

<main>
    <section>
        <h2>Informations personnelles</h2>
        <table border="1" cellpadding="5" cellspacing="0" width="80%" align="center">
            <tr><th>Champ</th><th>Valeur</th></tr>
            <tr><td>Ville</td><td>Bafoussam</td></tr>
            <tr><td>Date de naissance</td><td>12/05/2008</td></tr>
            <tr><td>Email</td><td><a href="mailto:jean.mbarga@eleve.cm">jean.mbarga@eleve.
cm</a></td></tr>
            <tr><td>Téléphone</td><td>+237 6XX XX XX XX</td></tr>
            <tr>
                <td colspan="2" style="text-align:center; font-style:italic;">
                    Représentant légal : M. Mbarga Paul
                </td>
            </tr>
        </table>
    </section>

    <section>
        <h2>Compétences TI</h2>
        <ul>
            <li>Algorithmique
                <ol>
                    <li>Tableaux</li>
                    <li>Tris (Bulles, Sélection, Insertion)</li>
                    <li>Recherche (Séquentielle, Dichotomique)</li>
                </ol>
            </li>
            <li>Web Statique
                <ol>
                    <li>HTML5 (Structure, Sémantique, Formulaires)</li>
                    <li>CSS3 (Boîtes, Flexbox, Grid, Responsive)</li>
                </ol>
            </li>
            <li>Maintenance
                <ol>
                    <li>Matériel (Montage, Diagnostic, Remplacement composants)</li>
                    <li>OS (Installation Windows/Linux, Drivers, Mises à jour)</li>
                    <li>Réseau (Câblage RJ45, Config IP, WiFi, Dépannage)</li>
                </ol>
            </li>
        </ul>
    </section>
</main>

<footer>
    <p>&copy; 2025 Mbarga Jean – Lycée de Bafoussam</p>
    <p><a href="mailto:jean.mbarga@eleve.cm">Me contacter par email</a></p>
</footer>
</body>
</html>

```

**Points clés du corrigé :**

- **Chemins relatifs corrects depuis pages/ :** ../images/, ../css/, ../index.html, notes.html (même dossier).
- **Structure sémantique HTML5** (header, nav, main, section, footer).
- **Tableau équilibré :** 2 colonnes partout, colspan="2" sur la dernière ligne (une seule cellule).
- **Listes imbriquées valides :** <ol> à l'intérieur du <li> parent.
- **<span class="highlight">** pour le mot en ligne.
- **mailto:** dans le footer.

## 10 EXERCICES

### 10.1 Je vérifie mes connaissances

#### EXERCICE 8 QCM : à chaque question, une seule bonne réponse

Pour chaque question, recopie la lettre de la bonne réponse.

1. Le code correct pour déclarer l'encodage des caractères accentués dans une page HTML est :
  - (a) `<meta charset="utf-8">`
  - (b) `<charset value="utf-8">`
  - (c) `<meta encoding="accents">`
  - (d) `<html charset="utf-8">`
2. L'attribut qui précise l'adresse d'un hyperlien dans la balise `<a>` est :
  - (a) `src`
  - (b) `link`
  - (c) `href`
  - (d) `url`
3. Pour fusionner deux cellules d'un tableau sur une même ligne, on utilise l'attribut :
  - (a) `rowspan`
  - (b) `colspan`
  - (c) `cellpadding`
  - (d) `merge`
4. Dans un formulaire, pour que plusieurs boutons radio soient mutuellement exclusifs, ils doivent avoir :
  - (a) le même attribut `id`
  - (b) le même attribut `name`
  - (c) le même attribut `value`
  - (d) le même attribut `type`

**EXERCICE 9** Vrai ou faux — justifier

Réponds par vrai ou faux, puis justifie ta réponse en une phrase.

1. Une balise HTML s'écrit toujours par paire : une balise ouvrante et une balise fermante.
2. L'attribut `title` d'un hyperlien permet d'afficher une infobulle au survol de la souris.
3. La balise `<img>` possède une balise fermante `</img>`.
4. Un lien relatif `href="pages/contact.html"` suppose que le fichier `contact.html` se trouve dans le même dossier que la page courante.
5. La balise `<span>` est une balise de section de type en ligne, tandis que `<div>` est de type bloc.

**EXERCICE 10** Question de cours type Probatoire

Défins chacun des termes suivants et donne un exemple précis pour chacun : **balise**, **attribut de balise**, **hyperlien**, **URL**, **site web**.

**EXERCICE 11** Lecture de code

On donne l'extrait de code HTML suivant :

```
<ul>
  <li>Matières scientifiques
    <ol>
      <li>Mathématiques</li>
      <li>Physique</li>
    </ol>
  </li>
  <li>Matières littéraires</li>
</ul>
<a href="https://www.minesec.gov.cm" title="Site du MINESEC">MINESEC</a>
```

1. Quel type de liste est créé par la balise `<ul>` ? Par la balise `<ol>` ?
2. Comment appelle-t-on la structure formée par la liste `<ol>` placée à l'intérieur de la liste `<ul>` ?
3. Quel texte sera cliquable ? Que se passe-t-il lorsque l'utilisateur survole ce lien avec la souris ?
4. Le lien est-il relatif ou absolu ? Justifie.

## 10.2 Je m'entraîne

**EXERCICE 12** Chasse aux erreurs

Le code ci-dessous, écrit par un élève, contient exactement cinq erreurs. Recopie le code, repère chaque erreur, explique-la et corrige-la.

```
<html>
<head>
  <title>Mon lycee</title>
</head>
<body>
  <h1>Bienvenue au Lycee de Nkolbisson
  
  <a href="page/contact.html">Nous contacter</a>
  <form>
    <input type="radio" value="M"> Garcon
    <input type="radio" value="F"> Fille
  </form>
</body>
</html>
```

On précise que le fichier `contact.html` se trouve dans le dossier `pages` (et non `page`).

**EXERCICE 13** Tableau d'emploi du temps

Écris le code HTML complet d'un tableau représentant un emploi du temps simplifié : 5 colonnes (Lundi à Vendredi) et 4 lignes de créneaux horaires. Le tableau respectera les contraintes suivantes :

1. la première ligne contient les en-têtes de colonnes (jours), écrites avec la balise appropriée ;
2. une cellule « Pause » est fusionnée sur les 5 colonnes (troisième ligne) ;
3. une cellule « EPS » est fusionnée sur 2 lignes dans la colonne Vendredi ;
4. le tableau a une bordure visible, une largeur de 80 % de la page, un espacement intérieur des cellules de 5 pixels et un espacement entre cellules de 2 pixels.

**EXERCICE 14** Formulaire de commande de tenues scolaires

La coopérative du lycée veut commander les tenues en ligne. Écris le code HTML d'un formulaire contenant :

1. un champ de saisie « Nom et prénom » (texte) ;
2. une liste déroulante « Taille » proposant les valeurs S, M, L, XL ;
3. deux boutons radio « Sexe » : Masculin / Féminin (un seul choix possible) ;
4. deux cases à cocher « Options » : Broderie du nom, Ourlet ajusté ;

5. un bouton cliquable « Commander » de type submit et un bouton « Annuler » de type reset.

Chaque champ sera précédé d'une étiquette explicite.

#### EXERCICE 15 Chemins relatifs dans une arborescence

Le site du club informatique est organisé ainsi : le fichier `index.html` est à la racine du site ; l'image `photo.png` est dans le dossier `images` ; la page `contact.html` est dans le dossier `pages` ; les dossiers `images` et `pages` sont à la racine.

1. Dessine l'arborescence de ce site.
2. Écris la balise complète permettant d'insérer `photo.png` depuis `index.html`.
3. Écris la balise complète permettant d'insérer `photo.png` depuis `contact.html`.
4. Écris le lien hypertexte placé dans `index.html` pointant vers `contact.html`, puis le lien placé dans `contact.html` permettant de revenir à `index.html`.
5. Le lien `href="https://www.camtel.cm"` est-il relatif ou absolu ? Justifie.

### 10.3 Je me prépare au Probatoire

#### EXERCICE 16 Site vitrine d'un cybercafé de Bafoussam

M. Tchoupo gère le cybercafé « Net Express » à Bafoussam. Il te demande de créer un site web statique de présentation. L'arborescence prévue est la suivante : à la racine, `index.html` ; un dossier `images` contenant `logo.png` et `salle.jpg` ; un dossier `pages` contenant `tarifs.html`.

1. Définis les termes **site web** et **URL**, puis explique la différence entre un site web statique et un site web dynamique.
2. Écris le squelette complet de la page `index.html` (doctype, balises `html`, `head`, `body`), en veillant à ce que les caractères accentués s'affichent correctement.
3. Dans `index.html`, insère le logo `logo.png` avec un texte alternatif adapté, puis rends cette image cliquable vers la page `tarifs.html`.
4. Écris le code d'une liste à puces présentant trois services (Navigation Internet, Impression, Formation bureautique), le service « Formation bureautique » contenant lui-même une sous-liste numérotée de deux modules (Word, Excel).
5. Dans `tarifs.html`, écris le code d'un tableau de tarifs à bordure visible comportant 3 colonnes (Service, Durée, Prix en FCFA) et 3 lignes de données, la dernière ligne « Forfait étudiant : 500 FCFA la journée » étant fusionnée sur les 3 colonnes.

6. Écris, dans `tarifs.html`, le lien de retour vers `index.html` avec une infobulle « Retour à l'accueil ».

#### EXERCICE 17 Formulaire d'inscription à la bibliothèque scolaire

La bibliothèque du Lycée Classique de Dschang souhaite numériser ses fiches d'inscription. On te fournit la maquette suivante : le formulaire doit recueillir le nom complet (texte), la classe (liste déroulante : Seconde, Première, Terminale), le sexe (boutons radio), les types d'ouvrages souhaités (cases à cocher : Romans, Sciences, Histoire), un commentaire libre (zone de texte multiligne), et proposer un bouton « Valider » et un bouton « Effacer ».

1. Définis les termes **formulaire** et **attribut de balise**.
2. Écris le code HTML complet de ce formulaire, chaque champ étant associé à une étiquette.
3. Explique pourquoi les deux boutons radio doivent porter le même attribut `name`, et ce qui se passerait dans le cas contraire.
4. Quelle différence fais-tu entre un bouton de type `submit` et un bouton de type `reset` ?
5. Le proviseur demande que le formulaire envoie automatiquement les données dans une base de données du lycée. Explique pourquoi le HTML seul ne suffit pas, et nomme la technologie qui sera étudiée en Terminale pour réaliser ce traitement.
6. Cite deux autres limites du HTML et, pour chacune, la technologie qui permet de la pallier.

#### EXERCICE 18 Correction et enrichissement d'une page web

Une élève de Première TI a écrit la page suivante pour le site de sa classe :

```
<html>
<head><title>Classe de Premiere TI</title></head>
<body>
  <h1>Notre classe</h2>
  <p>Bienvenue sur la page de la classe de Premiere TI.</p>
  
  <table border="1">
    <tr><th>Jour</th><th>Matiere</th></tr>
    <tr><td>Lundi</td><td>Informatique</td>
  </table>
  <a href="https://www.minesec.gov.cm">Site officiel</a>
</body>
```

1. Recopie ce code et corrige les quatre erreurs qu'il contient (balise mal fermée, sépa-

rateur de chemin incorrect, attribut manquant sur l'image, tableau mal refermé).

2. Explique le rôle de l'attribut `alt` que tu as ajouté à l'image, et pourquoi il est indispensable.
3. Ajoute au code corrigé la déclaration permettant l'affichage correct des caractères accentués, en précisant où elle doit être placée.
4. Ajoute au lien hypertexte une infobulle « Ministère des Enseignements Secondaires ».
5. Structure la page en remplaçant le corps par deux sections : une balise `<section>` pour la présentation et une balise `<section>` pour l'emploi du temps, chacune précédée d'un titre de niveau 2.
6. L'élève veut que le titre principal apparaisse en bleu et centré. Explique pourquoi le HTML seul est insuffisant pour cela et nomme la technologie adaptée.

## 11 CORRIGÉS DES EXERCICES

## 12 LEÇON 05 : PROGRAMMATION WEB STATIQUE EN HTML — EXERCICES CORRIGÉS

### CORRIGÉ Exercice 1 — QCM : à chaque question, une seule bonne réponse

1. Réponse a. La déclaration correcte de l'encodage est `<meta charset="utf-8">`, placée dans l'en-tête `<head>`. L'encodage **UTF-8** permet l'affichage correct des caractères accentués (é, è, à, ç...). Les autres propositions n'existent pas en HTML : il n'y a ni balise `<charset>`, ni attribut `encoding` sur `<meta>`, ni attribut `charset` sur `<html>`.
2. Réponse c. L'attribut `href` (hypertext reference) de la balise `<a>` contient l'adresse (URL) de la destination du lien. L'attribut `src` sert, lui, à indiquer la source d'un média (image, son, vidéo).
3. Réponse b. L'attribut `colspan` fusionne des cellules horizontalement (sur plusieurs colonnes, donc sur une même ligne). `rowspan` fusionne verticalement (sur plusieurs lignes), et `cellpadding` définit l'espacement intérieur des cellules. L'attribut `merge` n'existe pas en HTML.
4. Réponse b. Des boutons radio forment un groupe mutuellement exclusif lorsqu'ils partagent le même attribut `name` : le navigateur n'autorise alors qu'une seule case cochée dans le groupe. L'attribut `id` doit au contraire être unique pour chaque élément, et `value` sert à définir la valeur envoyée au serveur.

### CORRIGÉ Exercice 2 — Vrai ou faux — justifier

1. Faux. La plupart des balises vont par paire (`<p>...</p>`), mais il existe des balises orphelines (ou **auto-fermantes** / void elements) qui n'ont pas de balise fermante : `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`.
2. Vrai. L'attribut `title` d'un lien (et de nombreuses autres balises) affiche une **infobulle** (tooltip) contenant son texte lorsque le pointeur de la souris survole l'élément.
3. Faux. `<img>` est une balise orpheline : elle ne possède pas de balise fermante `</img>`. Toute l'information est portée par ses attributs (`src`, `alt`, `width`...).
4. Faux. Le chemin `pages/contact.html` signifie que `contact.html` se trouve dans le sous-dossier `pages` du dossier contenant la page courante. S'il était dans le même dossier,

on écrirait simplement `href="contact.html"`.

5. Vrai. `<span>` est un conteneur en ligne (inline) : il n'occupe que la largeur de son contenu et ne provoque pas de retour à la ligne ; il sert à marquer une portion de texte. `<div>` est un conteneur de type bloc : il occupe toute la largeur disponible et crée un nouveau bloc.

### **CORRIGÉ** Exercice 3 — Question de cours type Probatoire

- **Balise** : marqueur de structuration du langage HTML, écrit entre chevrons (`<` et `>`), qui indique au navigateur la nature et le rôle d'un contenu. Exemple : `<h1>` ouvre un titre de niveau 1, fermé par `</h1>`.
- **Attribut de balise** : information complémentaire placée dans la balise ouvrante, sous la forme `nom="valeur"`, qui précise le comportement ou les propriétés de l'élément. Exemple : dans ``, `src` et `alt` sont des attributs.
- **Hyperlien** : élément cliquable d'une page web qui renvoie vers une autre page, un autre site ou un emplacement précis ; il est créé par la balise `<a>` et son attribut `href`. Exemple : `<a href="tarifs.html">Voir les tarifs</a>`.
- **URL** (Uniform Resource Locator) : adresse normalisée permettant de localiser une ressource sur le Web. Exemple : `https://www.minesec.gov.cm` (URL absolue) ; `pages/contact.html` (URL relative).
- **Site web** : ensemble cohérent de pages web reliées entre elles par des hyperliens, hébergé sur un serveur et accessible via une adresse. Exemple : le site vitrine du cyber-café « Net Express » composé de `index.html` et `tarifs.html`.

Points de vigilance (type Probatoire) : chaque définition doit être accompagnée d'un exemple précis et syntaxiquement correct ; confondre balise et attribut est une erreur éliminatoire fréquente.

### **CORRIGÉ** Exercice 4 — Lecture de code

1. `<ul>` (unordered list) crée une liste à puces (non numérotée) ; `<ol>` (ordered list) crée une liste numérotée (ordonnée : 1, 2, 3...).
2. Il s'agit d'une liste imbriquée : la liste numérotée `<ol>` est placée à l'intérieur d'un élément `<li>` de la liste à puces `<ul>`. On parle d'imbrication de listes.
3. Le texte cliquable est « MINESEC » (contenu de la balise `<a>`). Au survol de la souris, une infobulle affichant « Site du MINESEC » apparaît, grâce à l'attribut `title`.
4. Le lien est absolu, car l'attribut `href` contient une URL complète avec le protocole

(<https://>) et le nom de domaine ([www.minesec.gov.cm](http://www.minesec.gov.cm)) : il désigne une ressource externe au site, localisée de manière indépendante de la position de la page courante.

### **CORRIGÉ** Exercice 5 — Chasse aux erreurs

Les cinq erreurs sont les suivantes :

1. Balise `<h1>` non fermée : le titre « Bienvenue au Lycée de Nkolbisson » doit être refermé par `</h1>`, sinon tout le reste de la page serait affiché comme un titre de niveau 1.
2. Attribut mal orthographié : `scr` doit être corrigé en `src` (source) ; sans cela, l'image ne s'affiche pas.
3. Chemin incorrect : le fichier se trouvant dans le dossier `pages`, le lien doit être `href="pages/contact.html"` et non `page/contact.html` (erreur 404 garantie).
4. Attribut `name` manquant sur les boutons radio : sans `name` commun, les deux boutons peuvent être cochés simultanément. Il faut ajouter `name="sexe"` aux deux `<input>`.
5. Absence de la déclaration d'encodage : il manque `<meta charset="utf-8">` dans le `<head>`, ce qui risque de provoquer l'affichage de caractères illisibles à la place des accents.

Code corrigé (avec accents désormais pris en charge grâce à l'UTF-8) :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Mon lycée</title>
</head>
<body>
  <h1>Bienvenue au Lycée de Nkolbisson</h1>
  
  <a href="pages/contact.html">Nous contacter</a>
  <form>
    <input type="radio" name="sexe" value="M"> Garçon
    <input type="radio" name="sexe" value="F"> Fille
  </form>
</body>
</html>
```

Point de vigilance : en production réelle, on écrirait « Lycée », « Garçon » avec leurs accents, justement rendus possibles par la déclaration UTF-8 ajoutée.

### **CORRIGÉ** Exercice 6 — Tableau d'emploi du temps

```
<table border="1" width="80%" cellpadding="5" cellspacing="2" align="center">
```

```

<tr>
  <th>Heures</th><th>Lundi</th><th>Mardi</th>
  <th>Mercredi</th><th>Jeudi</th><th>Vendredi</th>
</tr>
<tr>
  <td>7h30 – 9h30</td><td>Maths</td><td>Physique</td>
  <td>Français</td><td>Anglais</td><td rowspan="2">EPS</td>
</tr>
<tr>
  <td>9h30 – 10h00</td><td colspan="4" align="center">Pause</td>
</tr>
<tr>
  <td>10h00 – 12h00</td><td>Informatique</td><td>Histoire</td>
  <td>Maths</td><td>SVT</td>
</tr>
</table>

```

#### Explications :

- Les en-têtes de colonnes utilisent la balise `<th>` (table header), qui centre et met en gras automatiquement ; une colonne « Heures » a été ajoutée pour libeller les créneaux.
- La cellule « Pause » porte `colspan="4"` : elle fusionne les 4 colonnes de jours disponibles (Lundi à Jeudi) sur la ligne concernée. La colonne Vendredi est occupée par la fusion verticale `rowspan="2"` de la cellule « EPS » démarrée sur la ligne précédente.
- La cellule « EPS » porte `rowspan="2"` : elle occupe deux lignes dans la colonne Vendredi ; la ligne suivante (Pause) ne contient donc pas de cellule Vendredi (4 cellules de données + 1 cellule Heures = 5 cellules au total sur cette ligne, au lieu de 6).
- `border="1"` rend la bordure visible, `width="80%"` fixe la largeur relative, `cellpadding="5"` l'espacement intérieur et `cellspacing="2"` l'espacement entre cellules.

**Point de vigilance :** après une fusion par `rowspan`, la ligne suivante doit contenir une cellule de moins pour la colonne concernée ; l'oublier (ou utiliser un `colspan` trop large) décale tout le tableau et produit un rendu imprévisible.

#### CORRIGÉ

#### Exercice 7 — Formulaire de commande de tenues scolaires

```

<form action="commande.html" method="post">
  <p>
    <label for="nom">Nom et prénom :</label>
    <input type="text" id="nom" name="nom">
  </p>
  <p>
    <label for="taille">Taille :</label>
    <select id="taille" name="taille">
      <option value="S">S</option>
      <option value="M">M</option>
    </select>
  </p>
</form>

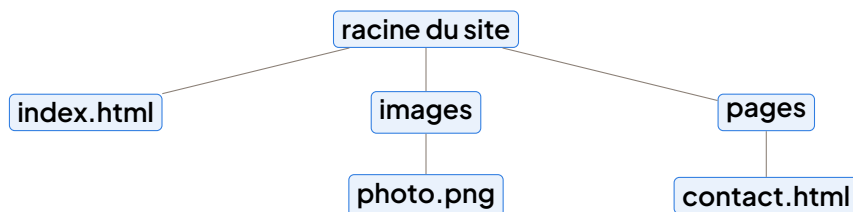
```

```
<option value="L">L</option>
<option value="XL">XL</option>
</select>
</p>
<p>
  Sexe :
  <input type="radio" id="masc" name="sexe" value="M">
  <label for="masc">Masculin</label>
  <input type="radio" id="fem" name="sexe" value="F">
  <label for="fem">Féminin</label>
</p>
<p>
  Options :
  <input type="checkbox" id="brod" name="broderie" value="oui">
  <label for="brod">Broderie du nom</label>
  <input type="checkbox" id="ourlet" name="ourlet" value="oui">
  <label for="ourlet">Ourlet ajusté</label>
</p>
<p>
  <input type="submit" value="Commander">
  <input type="reset" value="Annuler">
</p>
</form>
```

Remarques : les deux boutons radio partagent le même `name="sexe"`, ce qui garantit l'exclusivité du choix; les cases à cocher ont des `name` différents car elles sont indépendantes; chaque `<label>` est relié à son champ par le couple `for/id`, ce qui rend l'étiquette elle-même cliquable (amélioration de l'accessibilité et de l'ergonomie).

### **CORRIGÉ** Exercice 8 — Chemins relatifs dans une arborescence

#### 1. Arborescence du site :



Arborescence du site du club informatique

#### 2. Depuis `index.html` (à la racine), on descend dans le dossier `images` :

```

```

#### 3. Depuis `contact.html` (dans `pages`), il faut d'abord remonter d'un niveau avec `../` puis redescendre dans `images` :

```

```

#### 4. Liens de navigation :

```
<!-- Dans index.html : -->  
<a href="pages/contact.html">Nous contacter</a>  
<!-- Dans contact.html : -->  
<a href="../index.html">Retour à l'accueil</a>
```

5. Le lien `href="https://www.camtel.cm"` est absolu : il contient le protocole (`https://`) et le nom de domaine complet ; il pointe vers un site externe et ne dépend pas de l'emplacement de la page courante.

Point de vigilance : le séparateur de chemin sur le Web est toujours le slash / , jamais l'antislash \ de Windows.

### **CORRIGÉ** Exercice 9 — Site vitrine d'un cybercafé de Bafoussam

1. Site web : ensemble cohérent de pages web reliées par des hyperliens, hébergé sur un serveur et accessible via une adresse. URL : adresse normalisée permettant de localiser une ressource sur le Web (ex. `https://www.camtel.cm`). Un site statique affiche un contenu figé, identique pour tous les visiteurs, écrit uniquement en HTML/CSS : les pages ne changent que si l'on modifie le code source. Un site dynamique génère ses pages à la demande, à partir d'une base de données et d'un traitement côté serveur (contenu personnalisé, formulaires traités, comptes utilisateurs).
2. Squelette de `index.html` :

```
<!DOCTYPE html>  
<html lang="fr">  
<head>  
  <meta charset="utf-8">  
  <title>Net Express – Cybercafé de Bafoussam</title>  
</head>  
<body>  
  <h1>Bienvenue chez Net Express</h1>  
</body>  
</html>
```

La balise `<meta charset="utf-8">`, placée dans le `<head>`, garantit l'affichage correct des caractères accentués (é, è, à).

3. Logo inséré puis rendu cliquable en l'entourant d'une balise `<a>` :

```
<a href="pages/tarifs.html">  
    
</a>
```

## 4. Liste imbriquée des services :

```
<ul>
  <li>Navigation Internet</li>
  <li>Impression</li>
  <li>Formation bureautique
    <ol>
      <li>Word</li>
      <li>Excel</li>
    </ol>
  </li>
</ul>
```

## 5. Tableau de tarifs (dans tarifs.html) :

```
<table border="1" cellpadding="5" cellspacing="2">
  <tr><th>Service</th><th>Durée</th><th>Prix en FCFA</th></tr>
  <tr><td>Navigation</td><td>1 heure</td><td>200</td></tr>
  <tr><td>Impression N/B</td><td>1 page</td><td>100</td></tr>
  <tr><td colspan="3" align="center">Forfait étudiant : 500 FCFA la journée</td></tr>
</table>
```

## 6. Lien de retour avec infobulle :

```
<a href=" ../index.html" title="Retour à l'accueil">Accueil</a>
```

Le chemin ../index.html remonte du dossier pages vers la racine.

Barème indicatif (sur 10) : définitions et distinction statique/dynamique (2 pts) ; squelette avec meta charset (2 pts) ; image cliquable avec chemins relatifs corrects (2 pts) ; liste imbriquée (1,5 pt) ; tableau avec colspan (1,5 pt) ; lien retour avec title (1 pt). Points de vigilance : exiger ../ dans les liens depuis pages ; vérifier que l'image cliquable est bien à l'intérieur de la balise <a> ; la ligne fusionnée ne contient qu'une seule cellule.

## CORRIGÉ

## Exercice 10 — Formulaire d'inscription à la bibliothèque scolaire

- Formulaire** : zone d'une page web, délimitée par la balise <form>, permettant à l'utilisateur de saisir et d'envoyer des informations (champs de texte, boutons radio, cases à cocher, listes déroulantes, boutons). **Attribut de balise** : information complémentaire écrite dans la balise ouvrante sous la forme nom="valeur", qui précise les propriétés de l'élément (ex. type="text" dans <input>).

## 2. Code complet du formulaire :

```
<form action="inscription.html" method="post">
  <p>
    <label for="nom">Nom complet :</label>
    <input type="text" id="nom" name="nom">
  </p>
```

```
<p>
<label for="classe">Classe :</label>
<select id="classe" name="classe">
  <option value="2nde">Seconde</option>
  <option value="1ere">Première</option>
  <option value="Tle">Terminale</option>
</select>
</p>
<p>
  Sexe :
  <input type="radio" id="sxM" name="sexe" value="M">
  <label for="sxM">Masculin</label>
  <input type="radio" id="sxF" name="sexe" value="F">
  <label for="sxF">Féminin</label>
</p>
<p>
  Ouvrages souhaités :
  <input type="checkbox" id="rom" name="romans" value="oui">
  <label for="rom">Romans</label>
  <input type="checkbox" id="sci" name="sciences" value="oui">
  <label for="sci">Sciences</label>
  <input type="checkbox" id="his" name="histoire" value="oui">
  <label for="his">Histoire</label>
</p>
<p>
  <label for="com">Commentaire :</label> <br>
  <textarea id="com" name="commentaire" rows="4" cols="40"></textarea>
</p>
<p>
  <input type="submit" value="Valider">
  <input type="reset" value="Effacer">
</p>
</form>
```

3. Les deux boutons radio portent le même `name="sexe"` pour former un groupe : le navigateur applique alors l'exclusivité mutuelle (cocher l'un décoche l'autre), ce qui correspond à la réalité — un élève ne peut être que d'un seul sexe. Sans `name` commun, les deux boutons seraient indépendants et l'utilisateur pourrait cocher les deux simultanément, ce qui produirait une donnée incohérente.
4. Un bouton `submit` envoie les données du formulaire vers la page indiquée par l'attribut `action`; un bouton `reset` réinitialise tous les champs à leurs valeurs initiales, sans rien envoyer.
5. Le HTML seul ne sait que décrire l'interface du formulaire ; il ne peut ni recevoir, ni traiter, ni enregistrer les données dans une base de données : cela exige un traitement côté serveur. La technologie étudiée en Terminale pour cela est le PHP (couplé à un SGBD comme MySQL).
6. Deux autres limites du HTML :
  - Pas de mise en forme avancée (couleurs, polices, positionnement fin) : palliée par le CSS (Cascading Style Sheets) ;

- Pas d'interactivité côté client (calculs, vérification de saisie, réaction aux clics sans recharger la page) : palliée par le JavaScript.

Barème indicatif (sur 10) : définitions (1 pt); code complet et correct (4 pts); rôle du `name` commun (1,5 pt); distinction `submit/reset` (1 pt); limite et PHP (1,5 pt); deux autres limites (1 pt). Points de vigilance : la zone multiligne s'écrit `<textarea>` (et non `<input type="textarea">`); citer explicitement que le traitement se fait côté serveur; ne pas confondre CSS (présentation) et JavaScript (comportement).

### **CORRIGÉ** Exercice 11 — Correction et enrichissement d'une page web

#### 1. Les quatre erreurs corrigées :

- `<h1>` fermée par `</h2>` : incohérence de balises, on ferme par `</h1>`;
- chemin `images\classe.jpg` : sur le Web, le séparateur est le slash `/`, soit `images/classe.jpg`;
- attribut `alt` manquant sur `<img>` : il est obligatoire pour l'accessibilité et la validité;
- la dernière ligne du tableau `<tr>` n'est pas fermée : il manque `</tr>` avant `</table>`.

Code corrigé (incluant la déclaration d'encodage) :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Classe de Première TI</title>
</head>
<body>
  <h1>Notre classe</h1>
  <p>Bienvenue sur la page de la classe de Première TI.</p>
  
  <table border="1">
    <tr><th>Jour</th><th>Matière</th></tr>
    <tr><td>Lundi</td><td>Informatique</td></tr>
  </table>
  <a href="https://www.minesec.gov.cm">Site officiel</a>
</body>
</html>
```

2. L'attribut `alt` fournit un texte alternatif affiché si l'image ne peut pas être chargée (chemin erroné, connexion lente, navigateur texte); il est aussi lu par les lecteurs d'écran pour les malvoyants et utilisé par les moteurs de recherche pour l'indexation. Il est donc indispensable pour l'accessibilité (**accessibilité**) et la robustesse de la page.

3. On ajoute dans le `<head>`, idéalement juste après son ouverture (avant le `<title>`) :

```
<meta charset="utf-8">
```

## 4. Lien avec infobulle :

```
<a href="https://www.minesec.gov.cm"
  title="Ministère des Enseignements Secondaires">Site officiel</a>
```

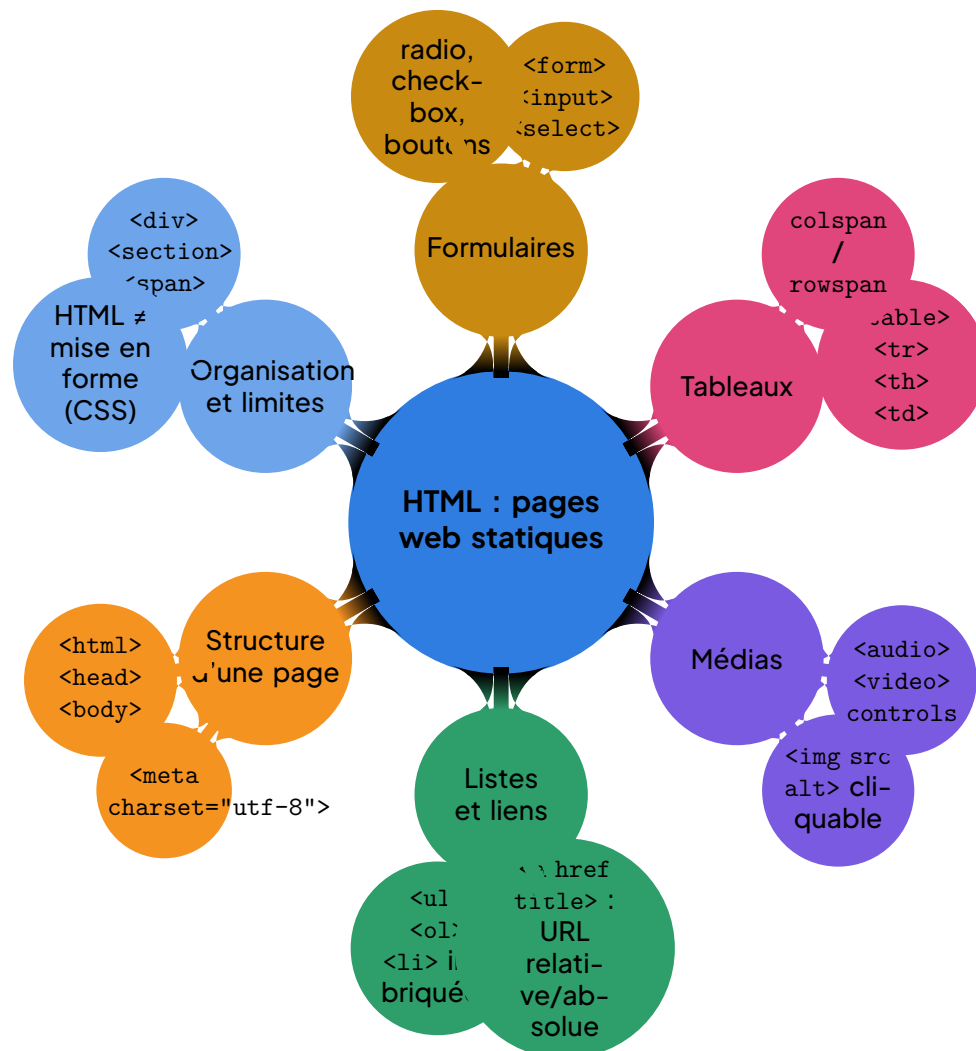
## 5. Page structurée en sections sémantiques :

```
<body>
  <section>
    <h2>Présentation</h2>
    <p>Bienvenue sur la page de la classe de Première TI.</p>
    
  </section>
  <section>
    <h2>Emploi du temps</h2>
    <table border="1">
      <tr><th>Jour</th><th>Matière</th></tr>
      <tr><td>Lundi</td><td>Informatique</td></tr>
    </table>
  </section>
  <a href="https://www.minesec.gov.cm"
    title="Ministère des Enseignements Secondaires">Site officiel</a>
</body>
```

## 6. Le HTML est un langage de structuration du contenu : il décrit la nature des éléments (titre, paragraphe, tableau) mais pas leur présentation. La couleur bleue et le centrage relèvent de la mise en forme, qui est du ressort du CSS (Cascading Style Sheets), langage de style complémentaire du HTML.

Barème indicatif (sur 10) : quatre erreurs corrigées (4 pts) ; rôle de alt (1,5 pt) ; meta charset bien placée (1 pt) ; infobulle (1 pt) ; sections (1,5 pt) ; limite du HTML et CSS (1 pt). Points de vigilance : le titre de niveau 2 à l'intérieur de chaque <section> est bien un <h2> (le <h1> reste unique en tête de page) ; exiger la justification « HTML = structure, CSS = présentation » pour la dernière question.

## 13 FICHE BILAN



Carte mentale de la leçon : les six compétences clés de la programmation web statique en HTML.

### FICHE BILAN L'essentiel à connaître pour l'examen

#### Définitions clés

- **Site web** : ensemble de pages web reliées par des hyperliens, hébergé sur un serveur et consultable via un navigateur.
- **Balise** : marqueur encadré par `<` et `>` qui structure le contenu ; la plupart vont par paires (`<p>...</p>`), certaines sont orphelines (`<br>`, `<img>`, `<input>`).
- **Attribut** : précision placée dans la balise ouvrante sous la forme `nom="valeur"` (ex. `src`,

href,width).

- **Hyperlien** : zone cliquable créée par `<a href="URL">` qui mène vers une autre page, un fichier ou une ancre.
- **URL** : adresse d'une ressource sur le Web ; absolue (complète, avec `http://`) ou relative (par rapport au dossier du site).
- **Formulaire** : zone de saisie (`<form>`) permettant à l'utilisateur d'entrer des données envoyées à un serveur.

Syntaxes à connaître par cœur

| Besoin                       | Code HTML                                                                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accents garantis             | <code>&lt;meta charset="utf-8"&gt;</code> dans <code>&lt;head&gt;</code>                                                                             |
| Lien avec infobulle          | <code>&lt;a href="page.html" title="Info"&gt;texte&lt;/a&gt;</code>                                                                                  |
| Image cliquable              | <code>&lt;a href="..."&gt;&lt;img src="photo.jpg" alt="..."&gt;&lt;/a&gt;</code>                                                                     |
| Son / vidéo                  | <code>&lt;audio src="..." controls&gt;;      &lt;video src="..." controls&gt;</code>                                                                 |
| Tableau                      | <code>&lt;table border="1"&gt; &lt;tr&gt; &lt;th&gt;...&lt;/th&gt; &lt;/tr&gt; &lt;tr&gt; &lt;td&gt;...&lt;/td&gt; &lt;/tr&gt; &lt;/table&gt;</code> |
| Fusion de cellules           | <code>colspan="n" (colonnes); rowspan="n" (lignes)</code>                                                                                            |
| Champ de saisie              | <code>&lt;input type="text" name="nom"&gt;</code>                                                                                                    |
| Bouton radio / case à cocher | <code>type="radio" (même name pour le groupe); type="checkbox"</code>                                                                                |
| Liste déroulante             | <code>&lt;select name="ville"&gt; &lt;option&gt;Douala&lt;/option&gt; &lt;/select&gt;</code>                                                         |
| Boutons                      | <code>type="submit", type="reset", type="button"</code>                                                                                              |
| Sections                     | <code>&lt;div&gt; (bloc générique), &lt;section&gt; (bloc thématique), &lt;span&gt; (en ligne)</code>                                                |

Méthodes express

- Créer une page : ouvrir le Bloc-notes ou un éditeur, écrire le squelette (`<!DOCTYPE html>`, `<html>`, `<head>` avec le charset, `<body>`), enregistrer en `.html`, ouvrir dans le navigateur.
- Organiser un site : un dossier racine contenant `index.html` et des sous-dossiers (`images/`, `css/`, `pages/`); les liens internes utilisent des URL relatives.
- Construire un tableau : d'abord les lignes `<tr>`, puis les cellules `<td>`; vérifier que chaque ligne a le même nombre de colonnes en comptant les fusions.
- Limites du HTML : pas de mise en forme élaborée (rôle du CSS), pas d'interactivité (rôle de JavaScript), pas de traitement côté serveur ; HTML décrit uniquement la structure du contenu.

## À RETENIR

## Checklist avant le Bac

- ☐ Je sais écrire de mémoire le squelette complet d'une page HTML avec `<meta charset="utf-8">`.
- ☐ Je distingue balise ouvrante, balise fermante et balise orpheline.
- ☐ Je sais imbriquer une liste dans une autre liste (`<ul>` dans un `<li>`).
- ☐ Je différencie une URL absolue d'une URL relative et j'ajoute une infobulle avec `title`.
- ☐ Je sais insérer une image avec `src` et `alt`, et la rendre cliquable.
- ☐ Je sais insérer un son et une vidéo avec l'attribut `controls`.
- ☐ Je construis un tableau avec `border`, `width`, `align`, `cellpadding`, `cellspacing`.
- ☐ Je fusionne des cellules avec `colspan` et `rowspan` sans décaler les lignes.
- ☐ Je crée un formulaire complet : champ texte, boutons radio, cases à cocher, liste déroulante, boutons `submit/reset`.
- ☐ Je sais organiser les dossiers d'un site et utiliser `<div>`, `<section>` et `<span>`.
- ☐ Je sais énoncer les limites du HTML (ni style, ni interactivité, ni traitement serveur).
- ☐ Je vérifie toujours que chaque balise ouverte est bien fermée et que les attributs sont entre guillemets.



Signé • OnBuch+

Ludovic A.  
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+